

大的区别。但在运输层和应用层是否也有重大差别呢？我们很容易认为这些差别是很小的，因为在有线和无线网络中的网络层均为上层提供了同样的尽力而为服务模式。类似地，如果在有线和无线网络中都是使用诸如 TCP 和 UDP 的协议提供运输层服务，那么应用层也应该保持不变。在某个方面我们的直觉是对的，即 TCP 和 UDP 可以（也确实）运行在具有无线链路的网络中。在另一方面，运输层协议（特别是 TCP）通常在有线和无线网络中有时会有完全不同的性能。这里，在性能方面区别是明显的，我们来研究一下其中的原因。

前面讲过，在发送方和接收方之间的路径上，一个报文段不论是丢失还是出错，TCP 都将重传它。在移动用户情况下，丢失可能源于网络拥塞（路由器缓存溢出）或者切换（例如，由于重路由选择报文段到移动用户新的网络接入点引入的时延）。在所有情况下，TCP 的接收方到发送方的 ACK 都仅仅表明未能收到一个完整的报文段，发送方并不知道报文段是由于拥塞，或在切换过程中，还是由于检测到比特差错而被丢弃的。在所有情况下，发送方的反应都一样，即重传该报文段。TCP 的拥塞控制响应在所有场合也是相同的，即 TCP 减小其拥塞窗口，如 3.7 节讨论的那样。由于无条件地降低其拥塞窗口，TCP 隐含地假设报文段丢失是由于拥塞而非出错或者切换所致。我们在 6.2 节看到，在无线网络中比特错误比在有线网络中普遍得多。当这样的比特差错或者切换丢失发生时，没理由让 TCP 发送方降低其拥塞窗口（并因此降低发送速率）。此时路由器的缓存的确可能完全是空的，分组可以在端到端链路中丝毫不受拥塞阻碍地流动。

研究人员在 20 世纪 90 年代早期到中期就认识到，由于无线信道的高比特差错率和切换丢失的可能性，TCP 的拥塞控制反应在无线情况下可能会有问题。有三大类可能的方法用于处理这一问题：

- 本地恢复。本地恢复方法的目标是在比特差错出现的当时和当地（如在无线链路中）将其恢复。如在 6.3 节学习的 802.11 ARQ 协议，或者使用 ARQ 和 FEC 的更为复杂的方法 [Ayanoglu 1995]）。
- TCP 发送方知晓无线链路。在本地恢复方法中，TCP 发送方完全不清楚其报文段跨越一段无线链路。另一种方法是让 TCP 发送方和接收方知道无线链路的存在，从而将在有线网络中发生的拥塞性丢包和在无线网络中发生的差错/丢包区分开，并且仅对有线网络中的拥塞性丢包采用拥塞控制。在假设端系统能够做出这种区分的情况下，[Balakrishnan 1997] 详细研究了多种类型的 TCP。[Liu 2003] 研究了在一个端到端路径中区分有线部分丢包和无线部分丢包的技术。
- 分离连接方法。在分离连接方法中 [Bakre 1995]，移动用户和其他端点之间的端到端连接被打断为两个运输层连接：一个从移动主机到无线接入点，一个从无线接入点到其他通信端点（我们假定它是有线的主机）。该端到端连接因此是由一个无线部分和一个有线部分级连形成的。经无线段的运输层能够是一个标准的 TCP 连接 [Bakre 1995]，或是一个特别定制运行在 UDP 上的差错恢复协议。[Yavatkar 1994] 研究了经无线连接使用运输层选择性重传协议。在 [Wei 2006] 中的测量报告指出了分离 TCP 连接广泛用于蜂窝数据网络中，通过使用分离 TCP 连接，上述问题的确能够有很大改进。

我们这里有关无线链路上的 TCP 的讨论是十分简要的。在无线网络中有关 TCP 挑战和解决方案的深入展望能够在 [Hanabali 2005; Leung 2006] 中找到。我们鼓励读者去查

阅这些文献以了解这个正在进行的研究领域的详情。

考虑过运输层协议后，我们接下来考虑无线和移动性对应用层协议的影响。这里一个重要的考虑是无线链路经常具有相对较低的带宽，如我们在图 6-2 中所见。因此，运行在无线链路尤其是蜂窝无线链路上的应用程序，必须将带宽作为稀有物品对待。例如，一个为在 3G 电话上运行的 Web 浏览器提供服务的 Web 服务器，就不能像为运行在有线连接的浏览器那样提供含有大量图片的内容。尽管无线链路的确为应用层提出一些挑战，它们具有的移动性同样使得一大批位置知晓和环境知晓应用成为可能 [Chen 2000; Baldauf 2007]。更一般地，无线和移动网络将在未来的泛在计算环境实现中起着重要作用 [Weiser 1991]。显然，在谈及无线和移动网络对网络应用及其协议的影响时，公平而论我们仅看到了冰山一角！

6.9 小结

无线网络和移动网络使电话发生了革命性变化，同时也对计算机网络界产生了日益深远的影响。伴随着它们对全球网络基础设施的随时、随地、无缝地接入，它们不仅使网络接入变得更加无所不在，而且催生了一组新的、令人兴奋的位置相关服务。考虑到无线网络和移动网络不断增长的重要性，本章关注用于支持无线和移动通信的原理、通用链路技术以及网络体系结构。

本章以对无线网络和移动网络的介绍开始，描述了由这种网络中通信链路的无线特性所引发的挑战和由这些无线链路带来的移动性之间的重要区别。这使我们能够更好地区分、识别和掌握每个领域中的关键概念。我们首先关注无线通信，在 6.2 节中考虑了无线链路的特征。在 6.3 节和 6.4 节中，我们研究了 IEEE 802.11 (WiFi) 无线 LAN 标准、两个 IEEE 802.15 个人区域网络 (蓝牙和 ZigBee)，以及 3G 和 4G 蜂窝因特网接入。然后我们将注意力转向移动性问题。在 6.5 节中我们区分了多种形式的移动性，不同的移动性面临不同的挑战，并且看到了不同的解决方案。我们考虑了移动结点的定位和路由选择问题，以及对那些动态地从一个网络接入点移到另一个网络接入点的移动用户的切换问题。在 6.6 节和 6.7 节中，我们分别考察了这些问题在移动 IP 标准和 GSM 中是如何处理的。最后，我们在 6.8 节中考虑了无线链路和移动性对运输层协议和网络应用的影响。

尽管我们用了整整一章来学习无线网络和移动网络，但全面探索这个令人兴奋和快速扩展的领域需要一整本书或更多书的篇幅。我们鼓励读者通过查阅在本章中提供的许多参考资料，对这一领域进行更深入的研究。

课后习题和问题



复习题

6.1 节

- R1. 一个无线网络运行在“基础设施模式”下是什么含义？如果某网络没有运行在基础设施模式下，那么它运行在什么模式下？这种运行模式与基础设施模式之间有什么不同？
- R2. 在 6.1 节的分类法中，所确定的四种无线网络类型各是什么？你已经使用的是这些无线网络类型中的哪一种？

6.2 节

- R3. 下列类型的无线信道损伤之间有什么区别：路径损耗、多径传播、来自其他源的干扰？
- R4. 随着移动结点离开基站越来越远，为了保证传送帧的丢失概率不增加，基站能够采取的两种措施是什么？

6.3 ~ 6.4 节

- R5. 描述 802.11 中信标帧的作用。
- R6. 是非判断：802.11 站在传输一个数据帧前，必须首先发送一个 RTS 帧并收到一个对应的 CTS 帧。
- R7. 为什么 802.11 中使用了确认，而有线以太网中却未使用？
- R8. 是非判断：以太网和 802.11 使用相同的帧格式。
- R9. 描述 RTS 门限值的工作过程。
- R10. 假设 IEEE 802.11 RTS 和 CTS 帧与标准的 DATA 和 ACK 帧一样长，使用 CTS 和 RTS 帧还会有好处吗？为什么？
- R11. 6.3.4 节讨论了 802.11 移动性，其中无线站点从一个 BSS 到同一子网中的另一个 BSS。当 AP 是通过交换机互连时，为了让交换机能适当地转发帧，一个 AP 可能需要发送一个带有哄骗的 MAC 地址的帧，为什么？
- R12. 在某蓝牙网络中的一个主设备和在 802.11 网络中的一个基站之间有什么不同？
- R13. 在 802.15.4 ZigBee 标准中超级帧的含义是什么？
- R14. 在 3G 蜂窝数据体系结构中，“核心网”的作用是什么？
- R15. 在 3G 蜂窝数据体系结构中，RNC 的作用是什么？在蜂窝语音网中 RNC 起什么作用？

6.5 ~ 6.6 节

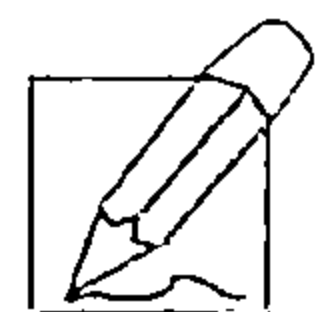
- R16. 如果某结点与因特网具有无线连接，则该结点必定是移动的吗？试解释之。假设一个使用膝上型电脑的用户携带电脑绕着她的住所散步，并且总是通过相同的接入点接入因特网。从网络的角度看，这是移动用户吗？试解释之。
- R17. 永久地址与转交地址之间的区别是什么？谁指派转交地址？
- R18. 考虑经移动 IP 的一条 TCP 连接。是非判断：在通信者和移动主机之间的 TCP 连接阶段经过该移动用户的归属网络，但数据传输阶段直接通过该通信者和移动主机，绕开了归属网络。

6.7 节

- R19. 在 GSM 网络中，HLR 和 VLR 的目的是什么？移动 IP 的什么要素类似于 HLR 和 VLR？
- R20. 在 GSM 网络中，锚 MSC 的作用是什么？

6.8 节

- R21. 为了避免单一无线链路降低一条端到端运输层 TCP 连接的性能，能够采取的三种方法是什么？



习题

- P1. 考虑在图 6-5 中单一发送方的 CDMA 例子。如果发送方的 CDMA 码是 $(1, -1, 1, -1, 1, -1, 1, -1)$ ，那么其输出（对于所显示的两个数据比特）是什么？
- P2. 考虑图 6-6 中的发送方 2，发送方对信道 $Z_{i,m}^2$ 的输出是什么（在它被加到来自发送方 1 的信号前）？
- P3. 假设在图 6-6 中的接收方希望接收由发送方 2 发送的数据。说明通过使用发送方 2 的代码，（经计算）接收方的确能够将发送方 2 的数据从聚合信道信号中恢复出来。
- P4. 在两个发送方、两个接收方的场合，给出一个包括 1 和 -1 值的两个 CDMA 编码的例子，不允许两个接收方从两个 CDMA 发送方提取出初始传输的比特。
- P5. 假设有两个 ISP 在一个特定的咖啡馆内提供 WiFi 接入，并且每个 ISP 有其自己的 AP 和 IP 地址块。
- a. 进一步假设，两个 ISP 都意外地将其 AP 配置运行在信道 11。在这种情况下，802.11 协议是否将完全崩溃？讨论一下当两个各自与不同 ISP 相关联的站点试图同时传输时，将会发生什么情况。

b. 现在假设一个 AP 运行在信道 1，而另一个运行在信道 11。你的答案将会有什么变化？

- P6. 在 CSMA/CA 协议的第 4 步，一个成功传输一个帧的站点在第 2 步（而非第 1 步）开始 CSMA/CA 协议。通过不让这样一个站点立即传输第 2 个帧（即使侦听到信道空闲），CSMA/CA 的设计者是基于怎样的基本原理来考虑的呢？
- P7. 假设一个 802.11b 站点被配置为始终使用 RTS/CTS 序列预约信道。假设该结点突然要发送 1000 字节的数据，并且所有其他站点此时都是空闲的。作为 SIFS 和 DIFS 的函数，并忽略传播时延，假设无比特差错，计算发送该帧和收到确认需要的时间。
- P8. 考虑在图 6-33 中显示的情形，其中有四个无线结点 A、B、C 和 D。这四个结点的无线电覆盖范围显示为其中的椭圆型阴影；所有结点共享相同的频率。当 A 传输时，仅有 B 能听到/接收到；当 B 传输时，A 和 C 能听到/接收到；当 C 传输时，B 和 D 能听到/接收到；当 D 传输时，仅有 C 能听到/接收到。

假定现在每个结点都有无限多的报文要向每个其他结点发送。如果一个报文的目的地不是近邻，则该报文必须要中继。例如，如果 A 要向 D 发送，来自 A 的报文必须首先发往 B，B 再将该报文发送给 C，C 则再将其发向 D。时间是分隙的，报文所用的传输时间正好是一个时隙，如在时隙 Aloha 中的情况一样。在一个时隙中，结点能够做下列工作之一：(i) 发送一个报文（如果它有报文向 D 转发）；(ii) 接收一个报文（如果正好一个报文要向它发送）；(iii) 保持静默。如同通常情况那样，如果一个结点听到了两个或更多的结点同时发送，出现冲突，并且重传的报文没有一个能成功收到。你这时能够假定没有比特级的差错，因此如果正好只有一个报文在发送，它将被位于发送方传输半径之内的站点正确收到。

- 现在假定一个无所不知的控制器（即一个知道在网络中每个结点状态的控制器）能够命令每个结点去做它（无所不知的控制器）希望做的事情，例如发送报文，接收报文，或保持静默。给定这种无所不知的控制器，数据报文能够从 C 到 A 传输的最大速率是什么，假定在任何其他源/目的地对之间没有其他报文？
- 现在假定 A 向 B 发送报文，并且 D 向 C 发送报文。数据报文能够从 A 到 B 且从 D 到 C 流动的组合最大速率是多少？
- 现在假定 A 向 B 发送报文且 C 向 D 发送报文。数据报文能够从 A 到 B 且从 C 到 D 流动的组合最大速率是多少？
- 现在假定无线链路由有线链路代替。在此情况下，重复问题 (a) ~ (c)。
- 现在假定我们又无线状态下，对于从源到目的地的每个数据报文，目的地将向源回送一个 ACK 报文（例如，如同在 TCP 中）。对这种情况重复问题 (a) ~ (c)。

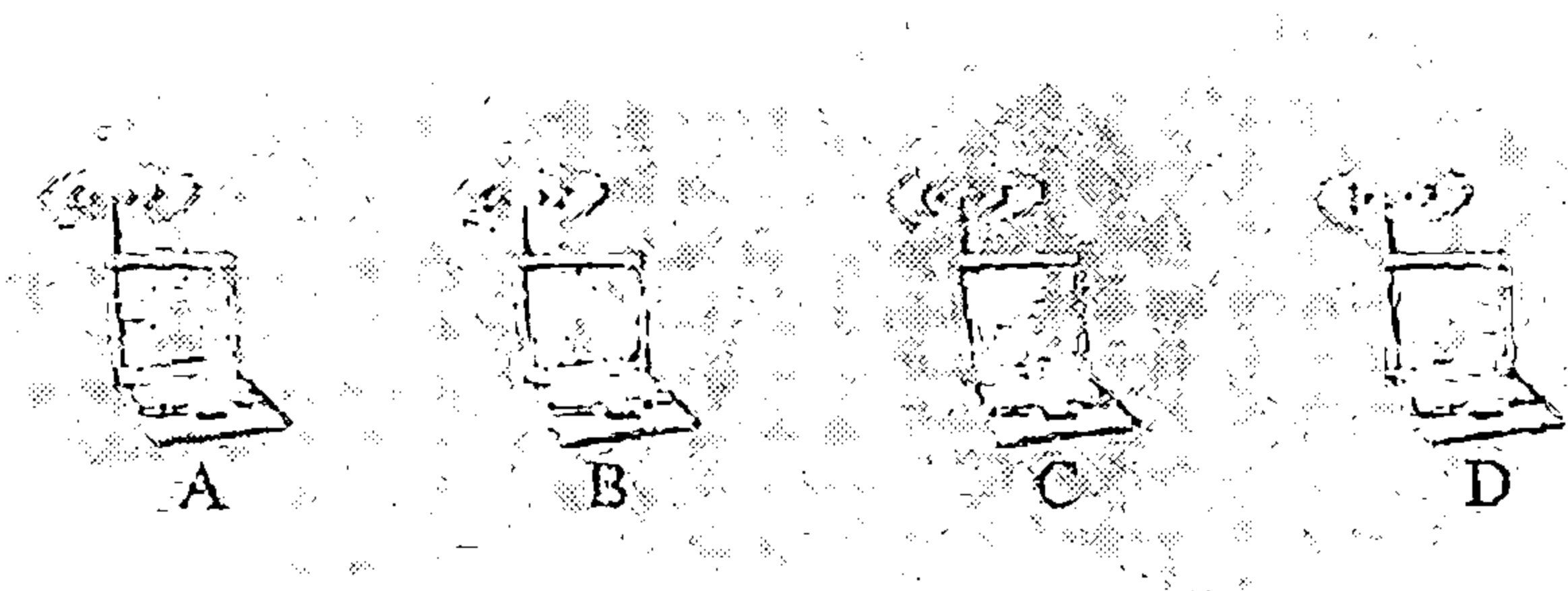
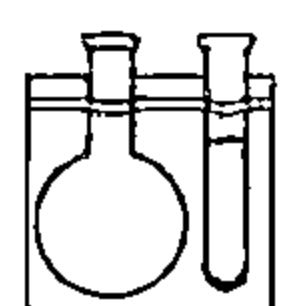


图 6-33 习题 P8 的情形

- P9. 描述 802.15.1 蓝牙帧的格式。你必须要阅读某些课外读物来获取这些信息。在帧格式中有哪些因素本质上会限制 802.15.1 网络中主动结点数量为 8 呢？试解释之。
- P10. 考虑下列理想化的 LTE 情形：下行信道（参见图 6-20）划分为时隙，使用了 F 个频率。有 4 个结点 A、B、C 和 D 分别以 10Mbps、5Mbps、2.5Mbps 和 1Mbps 速率在下行信道上可到达基站。这些速率假定基本在所有 F 个频率上能够利用所有时隙只向一个站点进行发送。基站具有无限量的数据向每个结点发送，并且在下行子帧中的任何时隙期间使用 F 个频率中的任何之一能够向这 4 个站点之一发送。

- a. 假定基站在每个时隙期间能够向它选择的任何结点发送，它能向结点发送的最大速率是多少？你的解决方案公平吗？解释并定义你所指“公平”的含义。
- b. 如果有公平要求，即每个站点在每秒期间必须收到等量的数据，在下行子帧期间基站（向所有结点）的平均传输速率是多少？
- c. 假定该公平性准则是在子帧期间任何结点至多能够接收任何其他结点两倍多的数据。在下行子帧期间基站（向所有结点）的平均传输速率是多少？解释你是如何得到答案的。
- P11. 在 6.5 节，一种允许移动用户在外部网络间移动过程中保持其 IP 地址不变的建议方案是，让外部网络通告一个到该移动用户高度特定的路由，并使用现有的路由选择基础设施在整个网络中传播这一信息。我们将扩展性作为一种关注因素。假设移动用户从一个网络移动到另一个网络后，新的外部网络通告一个到移动用户的特定路由，旧的外部网络丢弃其路由。考虑路由信息如何在一个距离向量算法中传播（尤其是对于跨越全球的网络间的域间路由选择情况）。
- a. 一旦外部网络开始通告其路由，其他路由器能否立刻将数据报路由选择到新的外部网络呢？
- b. 不同的路由器有可能认为移动用户位于不同的外部网络中吗？
- c. 讨论网络中其他路由器最终知道到达移动用户的路径所用的时间范围。
- P12. 假设图 6-22 中通信者是移动的。概述为了将数据报从初始移动用户路由选择到（现在移动的）通信者所需要的额外的网络层基础设施。如图 6-23 中那样，显示在初始移动用户和（现在移动的）通信者之间数据报的结构。
- P13. 在移动 IP 中，移动性将对数据报在源和目的地间的端到端时延有怎样的影响？
- P14. 考虑 6.7.2 节最后讨论的链的例子。假设一个移动用户访问外部网络 A、B 和 C，当通信者在外部网络 A 中时，它开始一条与移动用户的连接。列出在外部代理之间和外部代理与归属代理之间，当移动用户从网络 A 到网络 B 再到网络 C 的过程中的报文序列。然后，假设未执行链接，并且通信者（以及归属代理）必须被显式地告知移动用户转交地址的改变。列出在第二种情况下需要交换的报文序列。
- P15. 考虑在一个具有外部代理的外部网络中的两个移动结点。在移动 IP 中，这两个移动结点是否可能使用相同的转交地址？解释你的答案。
- P16. 在我们对 VLR 如何用移动用户当前位置信息更新 HLR 的讨论中，与 VLR 地址对 HLR 相比，提供 MSRN 所具有的优缺点各是什么？



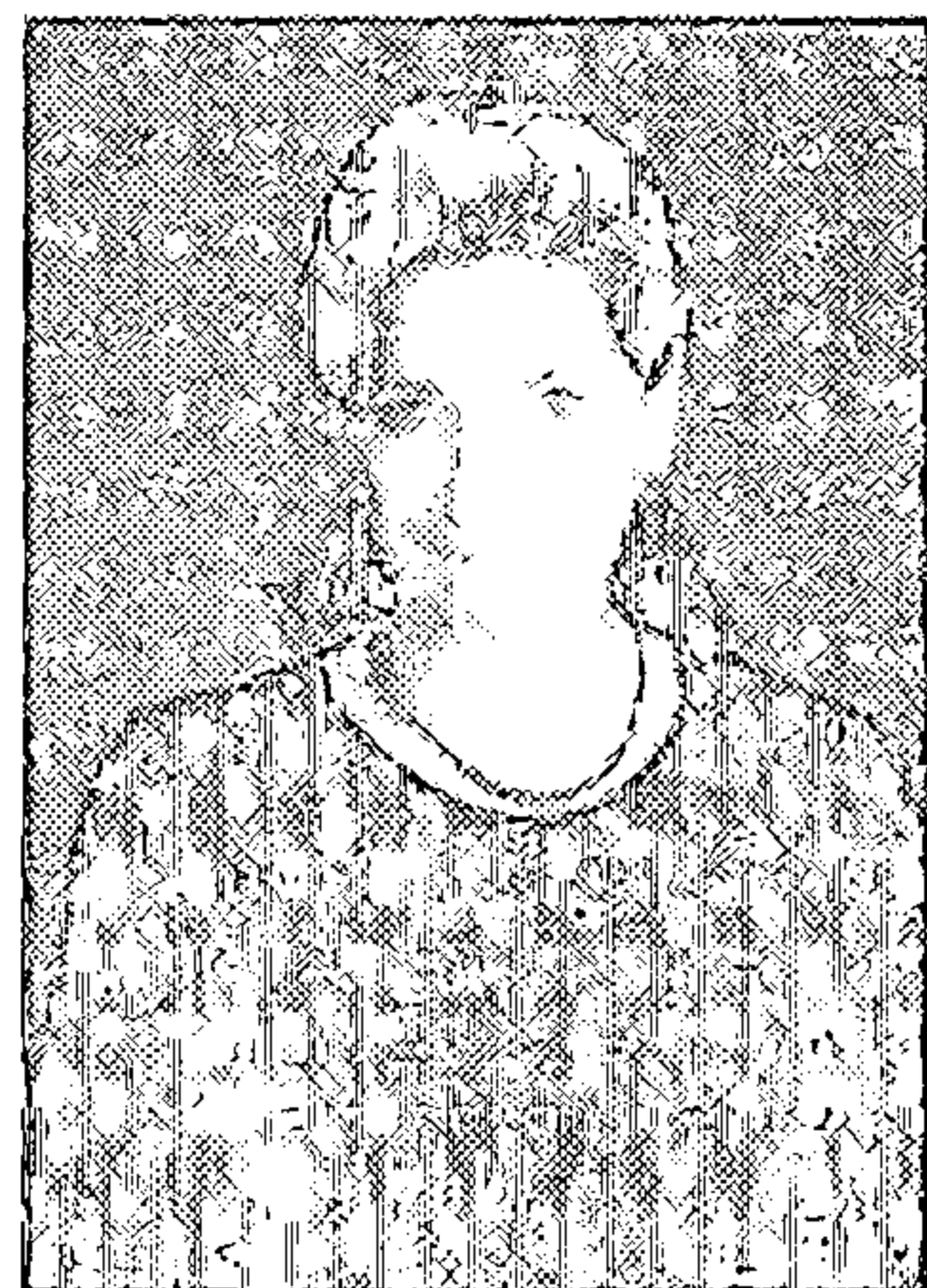
Wireshark 实验

在本书的配套 Web 站点上（<http://www.awl.com/kurose-ross>），你将会找到有关本章的一个 Wireshark 实验，该实验用于捕获和学习在无线便携机和接入点之间交换的 802.11 帧。



人物专访

Deborah Estrin 是加州大学洛杉矶分校的计算机科学教授，计算机网络的 Jon Postel 主席，嵌入式网络感知中心（CENS）的主任以及非营利组织 openmhealth.org 的联合奠基人。她从 MIT 获得了计算机科学博士学位（1985 年），从加州大学伯克利分校获得硕士学位（1980 年）。Estrin 的早期研究集中在包括多播和域间路由选择等在内的网络协议的设计。在 2002 年，Estrin 创建了美国国家自然科学基金资助的科学技术中心 CENS（<http://cens.ucla.edu>），以研发和探索环境监视技术和应用。当前 Estrin 及其同事正在研发供人分享感知（participatory sensing）系统，促进移动电话的可编程能力、邻近性和泛在性的发展；基本部署环境是移动健康（<http://openmhealth.org>）、社区数据收集和 STEM 教育（<http://mobilizingcs.org>）。Estrin 教授是美国艺术和科学研究院



Deborah Estrin

(2007 年) 和国家工程院 (2009 年) 的当选成员。她是 IEEE、ACM 和 AAAS 的会士。她被选为首名 ACM-W 雅典娜讲师 (2006 年), 获得 Anita Borg 学院的妇女远见创新奖 (2007 年), 被引入 WITI 名人纪念馆 (2008 年) 和获得来自 EPFL (2008 年) 和 Uppsala 大学 (2011 年) 的 Honoris Causa 博士。

◦ 请描述在您的职业生涯中从事的几件最为令人兴奋的项目?

20 世纪 90 年代中期当我在 USC 和 ISI 的时候, 非常荣幸地与像 Steve Deering、Mark Handley 和 Van Jacobson 这样的人物在一起工作, 设计多播路由选择协议 (特别是 PIM)。我试图将多播体系结构设计中的许多经验教训借鉴到生态监视阵列中, 这是我首次真正开始全身心地应用和多学科的研究。那让我对社会和技术领域中的创新感兴趣, 它们激发我对近期的研究领域——移动健康的研究兴趣。这些项目中的挑战随问题领域不同而不同, 但它们的共同之处是需要睁大我们的眼睛, 当我们在设计、部署、制作原型和试用之间重复时关注对问题的定义是否正确。没有一个问题能够借助于模拟或者构造的实验室实验加以分析解决。面对凌乱的问题和环境要保持清晰的体系结构, 它们都对我们的能力提出挑战, 并且它们都需要广泛的协作。

◦ 未来在无线网络和移动性方面您预见将会发生什么变化和创新?

我从来对预测未来不具太多信心, 但是随着智能手机变得越来越强大和因特网基本接入点增多, 我预言我们可能看到特色电话 (即那些不可编程和仅能用于语音和文本信息的电话) 的终结。我还认为我们将看到嵌入式 SIM 的继续迅速增长, 各种设备通过嵌入式 SIM 具有经过蜂窝网络以低数据率通信的能力。

◦ 您预见网络和因特网的未来向何处发展?

在命名数据和软件定义网络方面的努力将出现成果, 产生更可管理、可演化和更丰富的基础设施, 以及更一般地表现为推动体系结构的责任向协议栈较高层发展。在因特网开始时, 体系结构包括第四层及以下, 位于顶端的应用程序更为竖井式/独块式的。现在则是数据和分析支配着传输。

◦ 什么人对于您的职业生涯给予了激励?

有三个人出现在我的脑海中。第一个人是 Dave Clark, 他是因特网界的秘方和无名英雄。早期我有幸在他的左右, 看到他在 IAB 的“组织规范”和因特网管理方法方面所起的作用, 即大致共识和运行编码的引导者。第二个人是 Scott Shenker, 他的智慧才华、正直和坚持令我印象深刻。我努力但很难像他那样清晰地定义问题和给出解决方案。无论问题大和小, 我发电子邮件征求建议, 他总是第一个回复的人。第三个人是我的姐姐 Judy Estrin, 她将创造性和勇气投入到她的职业, 将想法和概念带入市场。没有 Judy 这类人, 因特网技术将不会改变我们的生活。

◦ 对于希望从事计算机科学和网络职业的学生, 您有什么推荐?

首先, 在你的学术工作中构建一个坚实的基础, 与你能够得到的任何、每个现实世界的工作经验相权衡。当你寻找一个工作环境时, 在你真正关心的问题领域寻找机会, 并且参与到你能够从中学习的思维敏捷的团队中。

多媒体网络

位于世界各个角落的人们当前正使用因特网来按需观看电影和电视节目。因特网电影和电视分发公司（如北美的 Netflix 和 Hulu、中国的“优酷”（Youku）和“看看”（Kankan））实际上已经成为家喻户晓的名称。而人们不仅观看因特网视频，他们也使用诸如 YouTube 这样的站点来上载和分发用户自己生成的内容，不仅成为因特网视频的消费者，也成为视频的生产者。此外，网络应用如 Skype、Google Talk 和 QQ（在中国十分流行），不仅允许人们经过因特网打“电话”，而且可以用视频和多方会议来强化该电话。事实上，我们能够确定地预测：到当前年代末，几乎所有的视频分发和语音会话都将在因特网的端到端发生，通常出现在经 4G 和 WiFi 接入网与因特网连接的无线终端上。

本章以 7.1 节中的多媒体应用的分类方法开始。我们将看到多媒体应用能够分为流式存储音频/视频、会话式 IP 音频/视频或流式实况音频/视频等几类。我们将看到这些应用类型中的每一类都有自己独特的服务需求，这些需求与传统的弹性应用如电子邮件、Web 浏览和远程注册的需求差异很大。在 7.2 节中，我们较为详细地研究流式视频。我们将探讨支撑流式视频的许多基础原则，包括客户缓存、预取和对可用带宽的适应性视频质量。我们也将研究内容分发网（Content Distribution Network, CDN），CDN 是今天广泛使用的最有影响的流式视频系统。在 7.3 节中，我们研究会话式语音和视频。它们不同于弹性应用，对端到端时延高度敏感，但能够容忍偶尔的数据丢失。此时我们将研究诸如适应性播放、前向纠错和差错掩盖等技术是如何减缓网络引入的丢包和时延的。我们还将考察 Skype 作为学习案例。在 7.4 节中，我们将学习 RTP 和 SIP，这是两个用于实时会话式语音和视频应用的协议。在 7.5 节中，我们将研究网络内部的一些机制，这些机制能用于区分一类流量（如会话式语音这样的时延敏感应用）和其他类型流量（如浏览 Web 网页这样的弹性应用），并且在多类流量中提供区分服务。

7.1 多媒体网络应用

我们将多媒体网络应用定义为任何应用音频或视频的网络应用。本小节中，我们将提供多媒体应用的分类法。我们将看到在该分类法中的每类应用都具有自己独特的服务要求和设计问题集合。但在深入讨论因特网多媒体应用前，考虑音频和视频媒体自身的内在特点是有用的。

7.1.1 视频的性质

视频最为显著的特点或许是它的高比特率（high bit rate）。经因特网分发的视频的典型传输速率从用于低质量视频会议的 100kbps 到用于流式高分辨率电影的 3Mbps。为了比较视频带宽需求与其他因特网应用的带宽需求的不同，我们简要地考虑三个不同的用户，他们每人使用了一种不同的因特网应用。第一位用户 Frank，他打算迅速将照片张贴到他的朋友的脸谱（Facebook）页面上。我们假设 Frank 每 10 秒钟查找一次新照片，并且这些

照片的平均大小是 200KB。（与以前一样，我们在整个讨论中都简单地假定 1KB = 8000 比特。）第二位用户 Martha 正从因特网（“云中”）向她的智能手机流式传输音乐。我们假定 Martha 正在听许多 MP3 歌曲，一首接着一首，都以 128kbps 速率进行编码。第三位用户 Victor 则正在观看以 2Mbps 编码的视频。最后，我们假设所有三位用户的会话长度是 4000 秒（大约 67 分钟）。表 7-1 比较了这三位用户的比特率和传输的总字节。我们看到这时流式视频消耗了最多的带宽，其比特率比脸谱和流式音乐应用的带宽大 10 倍。因此，当设计网络视频应用时，我们心中必须记住的第一件事是视频的高比特率需求。鉴于视频的流行性及其高比特率，也许不会对思科公司以下的预测感到惊讶 [Cisco 2011]：到了 2015 年，流式视频和存储视频将大约占全球因特网流量消费的 90%。

表 7-1 三种因特网应用的比特率需求的比较

	比特率	67 分钟传输的字节
Frank 脸谱	160kbps	80MB
Martha 音乐	128kbps	64MB
Victor 视频	2Mbps	1GB

视频的另一种重要特点是它能被压缩，因而要在视频质量与比特率间进行折中。视频是一个图像序列，图像通常以恒定的速率显示，例如每秒 24 幅或 30 幅图像。一个没有压缩、数字编码的图像由像素阵列组成，每个像素被编码为一定数量的比特来表示亮度和颜色。在视频中有两种类型的冗余，它们都可以用来进行视频压缩（video compression）。空间冗余是给定图像的内部冗余。从直觉上讲，一个主要由空白组成的图像具有高度的冗余，能够有效地压缩而不会明显降低图像质量。时域冗余反映一幅图像和后续图像的重复程度。例如，如果一幅图像和后续图像完全一致，没有理由对后续图像再进行编码；相反，在编码过程中直接指出后续图像是完全一样的则更为有效。今天商用的压缩算法能够将视频压缩为所希望的任何基本比特率。当然，比特率越高，图像质量越好，总体用户视觉体验也越好。

我们也能够使用压缩来生成相同视频的多重版本（multiple version），每个版本有不同的质量等级。例如，我们能够使用压缩生成相同视频的三个版本，速率分别为 300kbps、1Mbps 和 3Mbps。用户则能够根据他们的当前可用带宽来决定要观看哪个版本。具有高速因特网连接的用户可以选择 3Mbps 的版本；使用 3G 智能手机观看视频的用户可以选择 300kbps 的版本。类似地，在视频会议应用中的视频能被“动态”（on-the-fly）地压缩，以在会话用户之间给定的可用端到端带宽上提供最好的视频质量。

7.1.2 音频的性质

数字音频（包括数字化语音和音乐）的带宽需求比视频低得多。然而，数字音频具有自己独特的性质，当设计多媒体应用时必须考虑这些性质。为了理解这些性质，我们首先考虑模拟音频（由人和乐器所产生）是如何转换为数字信号的：

- 模拟音频信号首先以某种固定速率采样，例如每秒 8000 个样本。每个采样值是一个任意的实数。
- 然后每个采样值被“四舍五入”为有限个数值中的一个。这种操作被称为量化（quantization）。这些有限个数值（称为量化值）通常是 2 的幂，例如 256 个量化值。
- 每个量化值由固定数量的比特表示。例如，如果有 256 个量化值，那么每个值（因此每个音频采样）用一个字节来表示。所有样本的比特表示级联在一起就形成了该信号的数字表示。举例来说，如果一个模拟信号以每秒 8000 个样值采样，而且每个样本被量化并用 8 比特表示，则得到的数字信号的速率就为每秒 64 000 比

特。通过音频扬声器播放，这个数字信号则能够转换回来（也就是解码），形成一个模拟信号。然而，解码后的模拟信号仅是初始信号的近似，并且声音质量也许有明显的下降（例如，高频的声音可能在解码信号中丢失了）。通过增加采样速率和量化值的数量，解码信号能够更好地接近初始的模拟信号。因此（与视频一样），在解码信号的质量和比特率与数字信号存储空间之间存在一种折中。

我们刚才描述的基本编码技术称为脉冲编码调制（Pulse Code Modulation, PCM）。语音编码通常采用 PCM，采样速率为每秒 8000 个样本，每个样本用 8 比特表示，得到 64kbps 的速率。音频光盘（CD）也使用 PCM，采样速率为每秒 44 100 个样本，每个样本用 16 比特表示；这样使得单声道速率为 705.6kbps，立体声速率为 1.411Mbps。

然而，PCM 编码的语音和音乐很少在因特网中使用。与视频一样，取而代之的是使用压缩技术来减小流的比特速率。人类语音能被压缩到小于 10kbps 并仍然可懂。一种接近 CD 质量立体声音乐的流行压缩技术是 MPEG 1 第 3 层，更通常的叫法是 MP3。MP3 编码器通常能够压缩为许多不同的速率；128kbps 是最常使用的编码速率，并且能够产生非常小的声音失真。一种相关的标准是高级音频编码（Advanced Audio Coding, AAC），该标准已经随苹果公司而流行起来。与视频一样，能够以不同的比特率生成多重版本的预先录制的音频流。

尽管音频比特率通常比视频的比特率小得多，但用户通常对音频的小失误比视频的小失误更为敏感。例如，考虑在因特网上举行的视频会议。如果视频信号时不时地丢失几秒，该视频会议很可能继续进行而没有太多的用户抱怨。然而，如果音频信号经常丢失，用户就可能不得不中止该会话。

7.1.3 多媒体网络应用的类型

因特网能够支持各种各样有用的和娱乐性的多媒体应用。在本小节中，我们将多媒体应用分为三个大类：①流式存储音频/视频；②会话式 IP 语音/视频；③流式实况音频/视频。如我们很快将看到的那样，这些应用类型中的每种都有自己的服务需求和设计问题的集合。

1. 流式存储音频和视频

为使讨论具体化，我们这里聚焦流式存储视频，它通常结合了视频和音频组件。流式存储音频（例如流式音乐）非常类似于流式存储视频，尽管它的比特率通常要低得多。

在这类应用中，依赖的媒体是预先录制的视频（如电影、电视节目）预先录制的体育赛事或预先录制的用户生成的视频（如常在 YouTube 上看到的那些）。这些预先录制的视频放置在服务器上，用户向服务器发送请求按需观看视频。许多因特网公司今天提供流式视频，包括 YouTube（谷歌）、Netflix 和 Hulu。据估计，在今天的因特网接入网中流式存储视频构成了超过 50% 的下载流量 [Cisco 2011]。流式存储视频具有三个关键的不同特色。

- 流。在流式存储视频应用中，客户开始从服务器接收文件几秒之后，通常就开始播放视频。这意味着当客户正在从视频的一个位置开始播放时，与此同时正在从服务器接收该视频的后续部分。这种技术被称为流（streaming），它避免了在开始播放之前必须下载整个视频（并且引起一个潜在的长时延）。
- 相互作用。因为媒体是预先录制的，用户可以对多媒体内容进行暂停、重新配置前进、重新配置倒退、快进等操作。从一个客户提出这种请求到该动作在客户端表现出来，可接受的响应时间应该小于几秒。

- 连续播放。一旦视频开始播放，它应该根据初始记录的时序进行。因此，为了在客户端播放，必须从服务器中及时接收数据；否则，用户经历视频帧停滞（这时客户等待延迟的帧）或帧跳过（这时客户漏掉延迟的帧）。

到目前为止，对流式视频最重要的性能测量是平均吞吐量。为了提供连续的播放，网络为流式应用提供的平均吞吐量必须至少与该流视频本身的比特率一样大。如我们将在7.2节所见，通过使用缓存和预取，即使在吞吐量波动的时候，提供连续播放也是可能的，只要平均吞吐量（在5~10秒区间平均）保持在视频速率之上 [Wang 2008]。

对于许多流式视频应用，预先录制的视频被存储起来，并且从CDN而非从单一的数据中心流式播放。也有许多P2P视频流式应用，其中视频被存储在用户主机（对等方）上，不同视频块从可能分布在全球的不同对等方到达。在得知了因特网流式视频的性能后，我们将在7.2节更加深入地研究流式视频，特别关注客户缓存、预取、对可用带宽的适应性质量和CDN分发。

2. 会话式IP语音和视频

在因特网上的实时会话式语音通常称为因特网电话（Internet telephony），因为从用户的角度看，它类似于传统的电路交换电话服务。它也常被称为IP语音（Voice-over-IP, VoIP）。会话式视频与之类似，除了它包括参与者的语音以及视频外。今天的大多数语音和视频会话式系统允许用户生成具有三个或更多参与者的会议。会话式语音和视频广泛地应用于今天的因特网中，因特网公司Skype、QQ和Google Talk自称每天都有数亿用户。

在第2章有关应用服务需求的讨论中（图2-4），我们确定了一些轴，服务需求可以根据它们分类。其中的两个轴（即定时考虑和数据丢失容忍度）对会话式语音和视频应用尤其重要。定时考虑是很重要的，因为音频和视频会话式应用是高度时延敏感（delay-sensitive）的。对于具有两个或更多交互讲话者的会话来说，从用户讲话或移动开始到该动作显现在其他端的时延应当小于几百毫秒。对于语音，小于150ms的时延不会被人类听者觉察到，150~400ms的时延能够被接受，当时延超过400ms时，即使不会使对话变得完全无法理解，也会使语音会话变得令人沮丧。

另一个方面，会话式多媒体应用容忍丢包（loss-tolerant），即偶尔的丢失只会在音频/视频回放时偶尔出现干扰信号，而且这些丢失经常可以部分或者全部地隐藏。这些时延敏感但容忍丢包的特性明显不同于那些弹性数据应用（如Web浏览、电子邮件、社交网络和远程注册等）的特性。对于这些弹性应用，长时延令人恼火，但并不是特别有害，然而传输数据的完全和完整性是首要的。我们将在7.3节中更加深入地探讨会话式语音和视频，特别关注适应性播放、前向纠错和差错掩盖是如何减缓网络引入的分组丢失和时延的。

3. 流式实况音频和视频

这种第三类应用类似于传统的电台广播和电视，只是它通过因特网来传输而已。这些应用允许用户接收从世界上任何角落发出的实况无线电广播和电视传输。今天有数以千计、遍及全球的无线电台和电视台正在因特网上广播内容。

实况是类似于广播的应用，它们经常有很多接收相同音频/视频节目的客户。尽管通过使用4.7节中描述的IP多播技术，能够有效地完成向多个接收方分发实况音频/视频，但是多播分发今天更多的是通过应用层多播（使用P2P网络或CDN）或通过多个独立的单播流来实现。由于使用流式存储多媒体，网络必须为每个实况多媒体流提供大于该视频

消耗速率的平均吞吐量。因为事件是直播的，尽管定时限制没有会话式语音那么严格，但时延也可能成为问题。从用户选择观看一个实况传输到播放开始，能够容忍的时延最多为 10 秒。我们在本书中将不涉及流式实况媒体，因为用于流式实况媒体的许多技术（如初始缓存时延、适应性带宽使用和 CDN 分发）都类似于流式存储媒体所使用的技术。

7.2 流式存储视频

对于流式视频应用，预先录制的视频放置在服务器上，用户向这些服务器发送请求按需观看这些视频。用户可能从开始到结束都在观看视频而没有中断它，也可能在视频结束前停止观看它，或者通过暂停、重新定位到后面或前面镜头来与视频交互。流式视频系统可分为三种类型：UDP 流（UDP streaming）、HTTP 流（HTTP streaming）和适应性 HTTP 流（adaptive HTTP streaming）。尽管在实践中所有这三种系统都在使用，但绝大多数今天的系统应用了 HTTP 流和适应性 HTTP 流。

所有这三种形式的视频流的共同特点是广泛使用了客户端应用缓存，以此来缓解变化的端到端时延和变化的服务器和客户之间可用带宽量的影响。对于流式视频（存储的和实况的），用户通常能够容忍在客户请求某视频与该流视频在客户端播放之间有几秒钟的初始小时延。所以，当视频开始到达客户时，客户不必立即开始播放，反而能够在应用程序缓存中建立该视频的储备。一旦该客户建立起几秒钟的“已缓存但尚未播放”的视频储备，客户就可以开始视频播放了。这种客户缓存（client buffering）具有两种重要的优点。第一，客户端缓存能够吸收服务器到客户时延中的波动。如果某特殊部分的视频数据延迟了，只要它在“接收到但尚未播放”的视频耗尽之前到达，这个长时延将不会被注意到。第二，如果服务器到客户带宽暂时低于视频消耗速率，用户能够继续享受连续的播放，只要客户应用缓存仍没有完全排尽。

图 7-1 显示了客户端的缓存。在这个例子中，假定视频以固定的比特率编码，因此每个视频块包含了能在相同固定时间量 Δ 区间播放的视频帧。服务器在 t_0 传输第一个视频块，在 $t_0 + \Delta$ 传输第二个视频块，在 $t_0 + 2\Delta$ 传输第三个视频块等等。一旦客户开始播放，为了重新产生初始录制视频的定时，每个块应当在前一个块之后播放 Δ 时间单元。第一个视频块于 t_1 时刻到达，第二个视频块于 t_2 时刻到达。第 i 块的网络时延是服务器传输该块

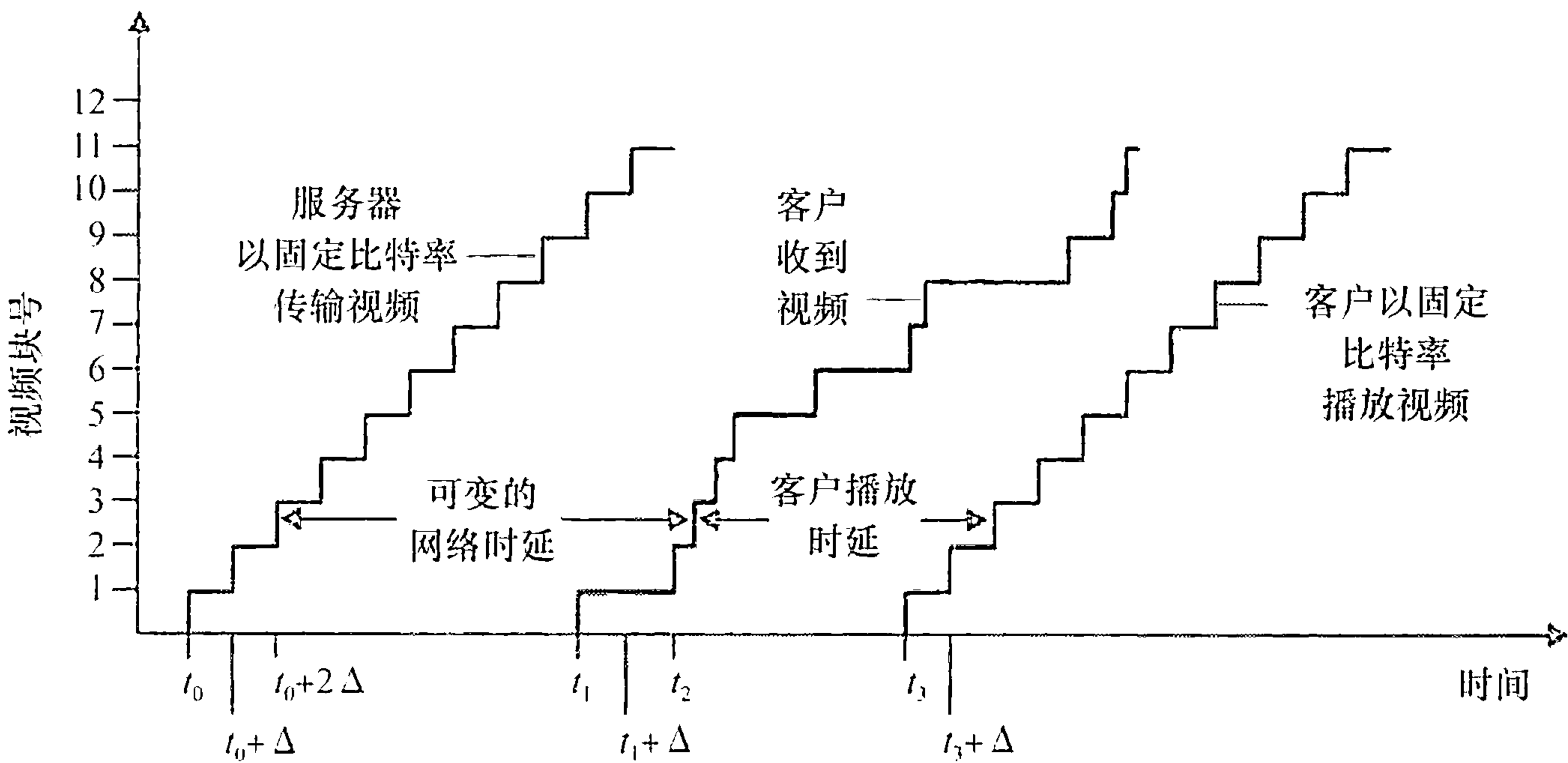


图 7-1 视频流中的客户播放时延

的时间与客户收到该块的时间之间的水平距离；注意到网络时延随视频块不同而变化。在此例子中，如果客户准备当第一块在 t_1 时刻一到达就开始播放，那么第二块将不能在 $t_1 + \Delta$ 时刻及时到达进行播放。在这种情况下，视频播放或将停止运行（等待第二块的到达）或可能漏掉第二块，即这两种情况都将导致不希望的播放损伤。相反，如果客户将播放延迟到 t_3 时刻开始，这时第一块到第六块都已经到达，所有已经收到的块在它们的播放时间前都能够进行周期性的播放。

7.2.1 UDP 流

我们这里仅简要讨论 UDP 流，在适当时向读者更为深入地介绍这些系统背后隐含的协议。使用 UDP 流，服务器通过 UDP 以一种稳定的速率记录下视频块，用与客户的视频消耗速率相匹配的速率传输视频。例如，如果视频消耗率是 2Mbps，每个 UDP 分组承载 8000 比特视频，则服务器将每隔 $(8000 \text{ 比特}) / (2 \text{ Mbps}) = 4 \text{ ms}$ 向其套接字发送一个 UDP 分组。如我们在第 3 章所知，因为 UDP 未采用某种拥塞控制机制，所以服务器能够以视频的消耗速率将分组推进网络中，而无 TCP 的速率控制的限制。UDP 流通常使用很小的客户端缓存，空间维持小于 1 秒视频就足够了。

在将视频块传递给 UDP 之前，服务器将视频块封装在运输分组中，该运输分组是专门为传输音频和视频而设计的，使用了实时传输协议（Real-Time Transport Protocol, RTP）[RFC 3550] 或某种类似（可能是专用）的方案。我们将在 7.3 节再讨论 RTP，那时我们将在会话式语音和视频系统环境中讨论 RTP。

UDP 流的另一种不同的性质是，除了服务器到客户的视频流外，两者间还并行地维护一个单独的控制连接，通过该连接，客户可发送有关会话状态变化的命令（如暂停、重新开始、重定位等）。这种控制连接在许多方面类似于我们在第 2 章中学习的 FTP 控制连接。在本书配套 Web 网站上更为详细地解释了实时流协议（Real-Time Streaming Protocol, RTSP）[RFC 2326]，它是一种用于这样的控制连接的流行开放协议。

尽管 UDP 流已经在多个开源系统和专用产品中得到应用，但它有三个重大不足。首先，由于服务器和控制之间的可用带宽无法预测并且是变化的，恒定速率 UDP 流不能够提供连续的播放。例如考虑以下场景：视频消耗速率为 1Mbps，服务器到客户可用带宽通常超过 1Mbps，但每过几分钟就有几秒钟时间其可用带宽低于 1Mbps。在这种场景下，以 1Mbps 恒定速率经 RTP/UDP 传输视频的 UDP 流系统很可能将提供不好的用户体验，在可用带宽低于 1Mbps 之后产生停滞或漏帧。UDP 流的第二个缺点是它要求如 RTSP 服务器这样的媒体控制服务器，以对每个进行中的客户会话处理客户到服务器的交互请求和跟踪客户状态（例如在视频中的客户播放点，视频是否被暂停或播放等）。这增加了部署大规模的按需视频系统的总体成本和复杂性。第三个缺点是许多防火墙配置为阻塞 UDP 流量，防止这些防火墙后面的用户接收 UDP 视频。

7.2.2 HTTP 流

在 HTTP 流中，视频直接作为具有一个特定 URL 的普通文件存储在 HTTP 服务器上。当用户要看视频时，客户和服务器之间建立一个 TCP 连接，并且发送一个对该 URL 的 HTTP GET 请求。服务器则尽可能快地在 HTTP 响应报文中发送该视频文件，这就是说，以 TCP 拥塞控制和流控制允许的尽可能快的速率进行处理。在客户端上，字节收集在一个

客户应用缓存中。一旦在缓存中字节数量超过了预先设定的阈值，该客户应用程序开始播放，具体而言，它周期性地从客户应用缓存中抓取视频帧，对帧解压缩并在用户屏幕上显示它们。

我们在第3章学习过，当通过TCP传输一个文件时，由于TCP的拥塞控制机制，服务器到客户的传输速率可能变化很大。特别是，传输速率以与TCP拥塞控制相关联的“锯齿”形（例如，图3-53）变化并非罕见。此外，分组也能由于重传机制而被大大延迟。因为TCP的这些特点，在20世纪90年代大多数人关于会话式的看法是流式视频将不可能在TCP上很好地工作。然而，随着时间的推移，流式视频系统的设计者知道了当使用了客户缓存和预取（在下面讨论）技术时，TCP的拥塞控制和可靠数据传输机制并不一定会妨碍连续播放。

在TCP上使用HTTP也使得视频穿越防火墙和NAT（它们常常被配置为阻挡UDP流量但允许大部分HTTP流量通过）更为容易。HTTP流消除了因需要媒体控制服务器（如RTSP服务器）带来的不便，减少了在因特网上大规模部署的成本。由于所有这些优点，今天的大多数流式视频应用（包括YouTube和Netflix）都使用HTTP流（在TCP上）作为它的底层流式协议。

1. 预取视频

我们刚才学习了客户端缓存可用于缓解变化的端到端时延和变化的可用带宽的影响。在前面图7-1的例子中，服务器以视频播放的速率传输。然而，对于流式存储视频，客户能够尝试以高于消耗速率的速率下载视频，因此预取（prefetching）将来会被消耗的视频帧。该预取的视频当然存储在客户应用缓存中。这样的预取自然伴随TCP流出现，因为TCP拥塞避免机制将试图使用服务器和客户之间的所有可用带宽。

为了深入洞察预取技术，我们来举个简单的例子。假设视频消耗速率是1Mbps，而网络从服务器到客户能够以恒定的1.5Mbps速率交付视频。客户则不仅能够以非常小的播放时延播放该视频，而且还能够以每秒500Kb的量增加缓存的视频数据。以这种方式，如果后来该客户在一段短暂时间内以小于1Mbps的速率接收数据，该客户由于在其缓存中的储备将能够继续提供连续的播放。[Wang 2008]显示了当平均TCP吞吐量大致为媒体比特率的两倍时，TCP流导致最小的饥饿和低缓存时延。

2. 客户应用缓存和TCP缓存

图7-2说明了客户和服务器之间HTTP流的交互。在服务器侧，视频文件中的白色部分已经通过服务器的套接字进行发送，而黑色部分是留下待发送的部分。在“通过套接字的门传送”之后，放置在TCP发送缓存中的字节在被传输进因特网之前如第3章所描述。在图7-2中，因为TCP发送缓存显示为满，服务器瞬间防止从视频文件发送更多的字节到套接字。在客户侧，客户应用程序（媒体播放器）从TCP接收缓存（通过其客户套接字）读出字节并将字节放入客户应用缓存中。与此同时，客户应用程序周期性地从客户应用缓存中抓取视频帧，解压缩并显示在用户屏幕上。注意到如果客户应用缓存大于该视频文件，则从服务器存储器到客户应用缓存移动字节的整个过程等价于普通文件经HTTP的下载过程，即客户直接将视频用TCP允许的尽可能快的速率从服务器中拉出来。

现在考虑在流播放期间当用户暂停视频时将发生的现象。在暂停期间，比特未从客户应用缓存中删除，甚至比特继续从服务器进入缓存。如果客户应用缓存是有限的，它可能

最终会变满，这将反过来引起对服务器的“反向压力”。具体而言，一旦客户应用缓存变满，字节不再从客户 TCP 接收缓存中删除，因此它也会变满。一旦客户 TCP 接收缓存变满，字节不再从服务器 TCP 发送缓存删除，因此它也变满。一旦客户 TCP 发送缓存变满，服务器不能向套接字中发送任何更多的字节。因此，如果用户暂停视频，服务器可能被迫停止传输，在这种情况下服务器被阻塞，直到用户恢复该视频。

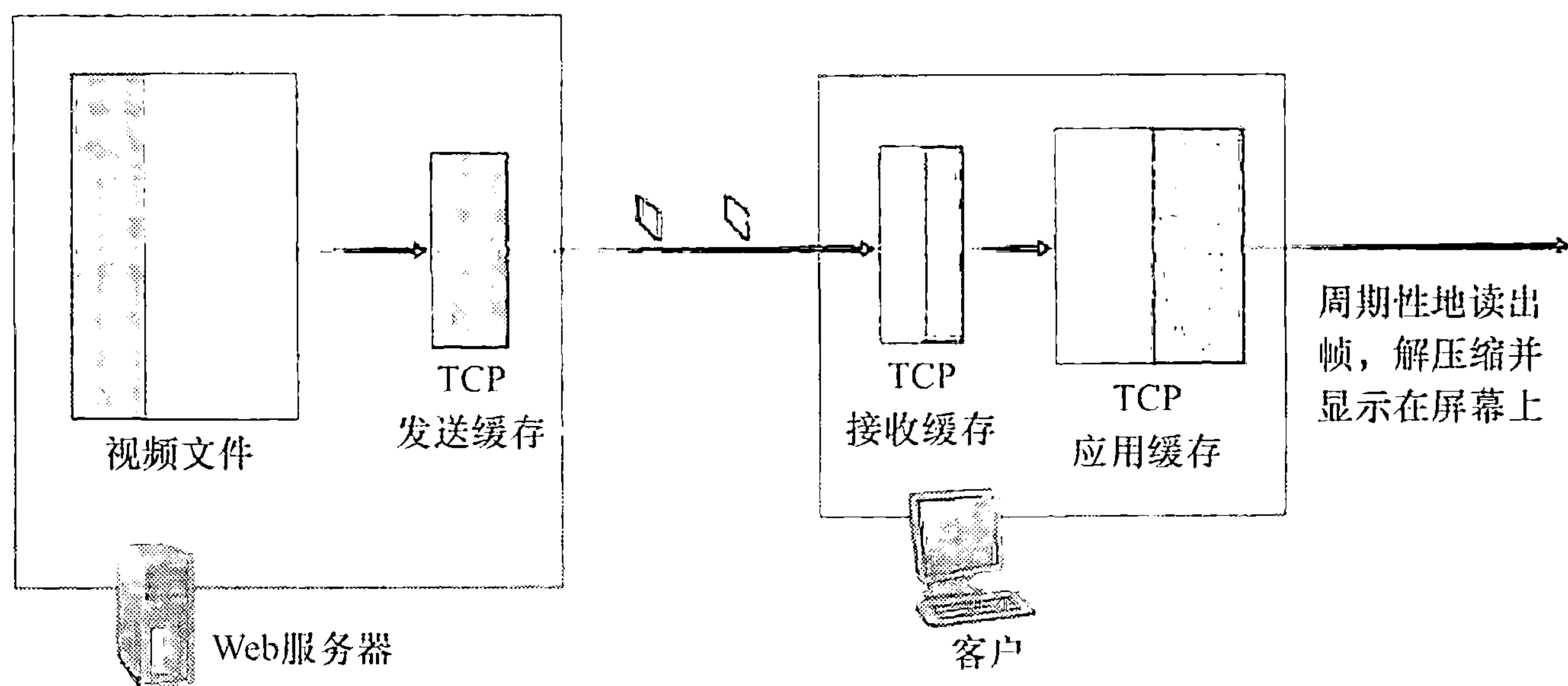


图 7-2 经 HTTP/TCP 的流式存储视频

事实上，甚至在常规的播放过程中（即没有暂停），如果客户应用缓存变满，反向压力将引起 TCP 缓存变满，这将迫使服务器降低其速率。为了决定其产生的速率，注意到当客户缓存删除 f 比特，它在客户应用缓存中产生了 f 比特的空间，这依次允许服务器发送额外的 f 比特。因此，服务器发送速率不能比客户端视频消耗速率更高。因此，当使用 HTTP 流时，一个满的客户应用缓存间接地对服务器到客户能够发送的视频速率施加了限制。

3. 流式视频的分析

某些简单的建模将有助于洞察由于应用缓存消耗所产生的初始播放时延和停滞。如图 7-3 所示， B 表示客户应用缓存的长度（以比特计）， Q 表示在客户应用缓存开始播放之前必须被缓存的比特数量。（当然， $Q < B$ 。） r 表示视频消耗速率，即客户在播放期间从客户应用缓存提取比特的速率。在此情况下，举例来说，如果视频的帧速率是 30 帧/秒，每（压缩）帧是 100 000 比特，则 $r = 3\text{Mbps}$ 。为了从细节看整体，我们将忽略 TCP 的发送和接收缓存。

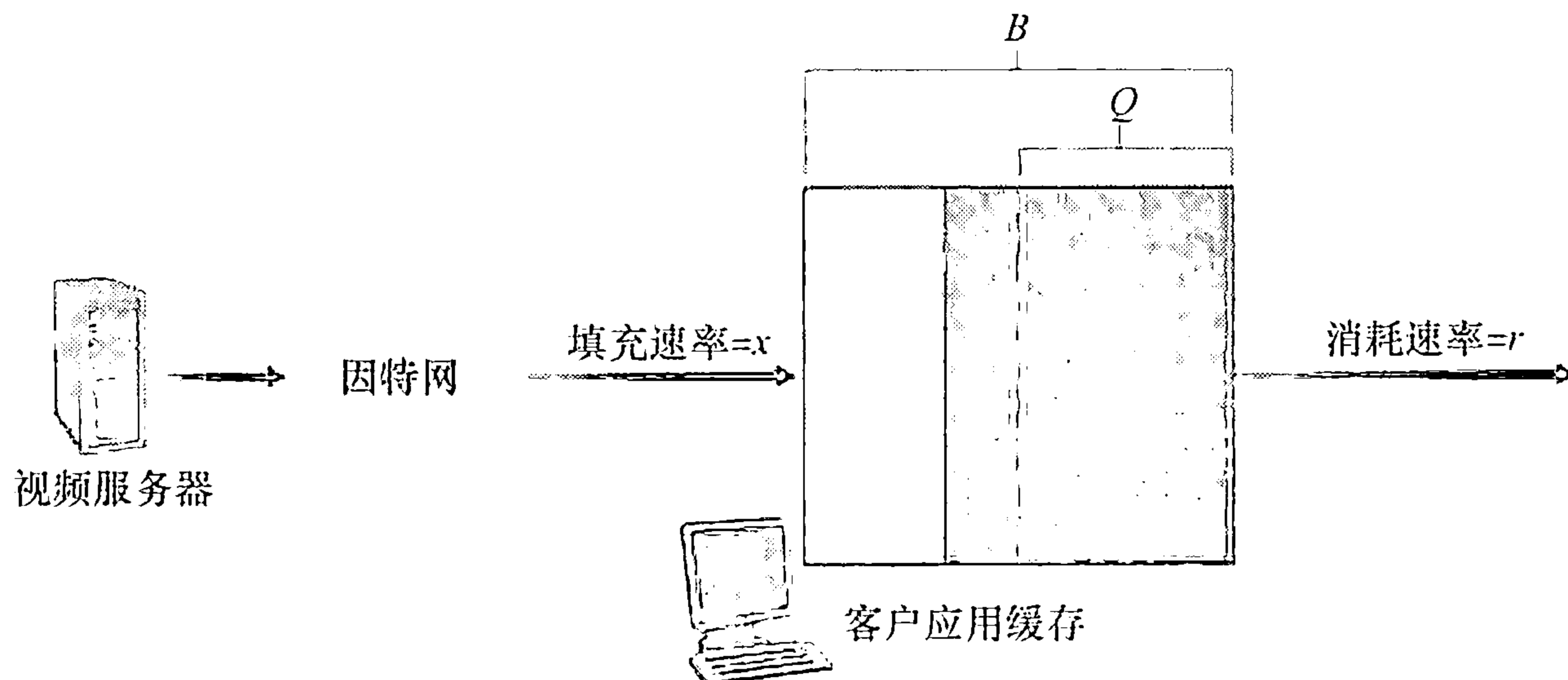


图 7-3 用户流式视频的客户端缓存的分析

我们假设无论何时客户缓存为非空时，服务器以一种恒定速率 x 发送比特。（这是一种显而易见的简化，因为 TCP 的发送速率由于拥塞控制而变化；在本章后面的习题中我们将考察更为真实的与时间相关的速率 $x(t)$ 。）假设在时刻 $t=0$ ，应用缓存为空，视频开始到达客户应用缓存。我们现在问，在什么时刻 $t=t_p$ 开始播放呢？并且在播放过程中，什么时刻 $t=t_f$ 客户应用缓存变满呢？

首先，我们来确定 t_p ，此时 Q 比特已经进入应用缓存并且开始播放。前面讲过比特以速率 x 到达客户应用缓存，并且在开始播放之前没有比特从其缓存中删除。所以，建立 Q 比特所需的时间（初始缓存时延）是 $t_p = Q/x$ 。

我们现在来决定 t_f ，这是客户应用缓存变满的时刻。我们先观察，如果 $x < r$ （即如果服务器发送速率小于视频消耗速率），则客户缓存将决不会满！的确，时刻 t_p 开始，缓存将以速率 r 排空并且仅以速率 $x < r$ 填充。最终客户缓存将完全排空，此时当客户缓存等待另一个 t_p 秒来建起 Q 比特的视频时，视频将在屏幕上停滞。所以，当网络中可用速率小于视频速率时，播放将在连续播放期和停滞播放期之间进行变动。在课后习题中，将请你决定每个连续播放期和停滞期的长度，它们都作为 Q 、 r 和 x 的函数。当 $x > r$ 时，现在我们来决定 t_f 。在这种情况下，在时刻 t_p 开始，缓存以 $x - r$ 的速率从 Q 增加到 B ，因为比特以速率 r 消耗但以速率 x 到达，如图 7-3 所示。有了这些提示，在课后习题中将请你决定 t_f ，即客户缓存变满的时刻。注意到当网络中的可用速率大于视频速率时，在初始缓存时延后，用户将享受连续的播放直到视频结束。

4. 视频的早期中止和重定位

HTTP 流系统经常利用 HTTP GET 请求报文中的 HTTP 字节范围首部（HTTP byte-range header），该首部指示了客户当前要从所希望的视频中获取的字节范围。当用户要在视频中及时重定位（即跳跃）到未来点时，这特别有用。当用户重定位到一个新位置时，客户发送一个新 HTTP 请求，用字节范围首部指出服务器应当从文件的哪个字节起发送数据。当服务器接收到该新的 HTTP 请求时，它能够忘记任何较早的请求，而是由字节范围请求中指示的字节开始发送。

在我们讨论重定位主题的时候，我们简要地提及当某用户重定位到视频中的某个未来点或提前终止视频时，某些由服务器发送的已预取但尚未观看的数据将不会被观看，即导致了网络带宽和服务器资源的浪费。例如，假设在视频中的某时刻 t_0 客户缓存充满 B 比特，在此时用户重定位到视频中的某个瞬间 $t > t_0 + B/r$ ，然后从这点起观察视频直到结束。在这种情况下，缓存中的所有 B 比特将未被观看，用于传输这 B 比特的带宽和服务器资源完全被浪费掉了。在因特网中，有大量的带宽因提前终止而浪费，这些成本可能相当大，特别是对于无线链路 [Ihm 2011]。由于这个原因，许多流系统仅使用了长度适当的客户应用缓存，或者将限制在 HTTP 请求中使用字节范围首部预取的视频数量 [Rao 2011]。

重定位和提前终止可以与下列做法类比：烹调了很多肉，仅吃了一部分，并将其他都扔掉，因而浪费了食物。因此，下次你因为没有吃完所有晚餐而被父母批评浪费食物时，你能够迅速反驳：当他们在因特网上观看电影并进行带宽重定位时，他们浪费了带宽和服务器资源！但是，别人错了不等于你对了，食物和带宽都不应被浪费！

7.2.3 适应性流和 DASH

尽管如前一小节所述，HTTP 流在实践中已经得到广泛部署（例如，YouTube 自发展

初期就开始部署), 但它具有严重缺陷, 即所有客户接收到相同编码的视频。尽管对不同的用户或者对于相同客户的不同时间而言, 为客户可用的带宽量有很大不同。这导致了一种新型基于 HTTP 的流的研究, 它常常被称为经 HTTP 的动态适应性流 (Dynamic Adaptive Streaming over HTTP, DASH)。在 DASH 中, 视频编码为几个不同的版本, 其中每个版本具有不同的比特率, 对应于不同的质量水平。客户动态地请求来自不同版本且长度为几秒钟的视频段数据块。当可用带宽量较高时, 客户自然地选择来自高速率版本的块; 当可用带宽量较低时, 客户自然地选择来自低速率版本的块。客户用 HTTP GET 请求报文一次选择一个不同的块 [Akhshabi 2011]。

一方面, DASH 允许客户使用不同的以太网接入速率流式播放具有不同编码速率的视频。使用低速 3G 连接的客户能够接收一个低比特率 (和低质量) 的版本, 使用光纤连接的客户能够接收高质量的版本。在另一方面, 如果端到端带宽在会话过程中改变的话, DASH 允许客户适应可用带宽。这种特色对于移动用户特别重要, 当移动用户相对于基站移动时, 通常他们能感受到其可用带宽的波动。例如, Comcast 已经部署一个适应性流系统, 其中每个视频源文件编码成 8 ~ 10 种不同的 MPEG-4 格式, 通过实施适应性来应对变化的网络和设备条件, 允许最高质量的视频格式以流形式发送给客户。

使用 DASH 后, 每个视频版本存储在 HTTP 服务器中, 每个都有一个不同的 URL。HTTP 服务器也有一个告示文件 (manifest file), 为每个版本提供了一个 URL 及其比特率。客户首先请求该告示文件并且得知各种各样的版本。然后客户通过在 HTTP GET 请求报文中对每块指定一个 URL 和一个字节范围, 一次选择一块。在下载块的同时, 客户也测量接收带宽并运行一个速率决定算法来选择下次请求的块。自然地, 如果客户缓存的视频很多, 并且测量的接收带宽较高, 它将选择一个高速率的版本。同样, 如果客户缓存的视频很少, 并且测量的接收带宽较低, 它将选择一个低速率的版本。因此 DASH 允许客户自由地在不同的质量等级之间切换。因为由改变版本引起的比特率突然下降可能会导致可觉察到的视觉质量下降, 所以可以使用多个中间版本来减小比特率, 从而平滑地迁移到客户消耗速率低于其可用接收带宽之下的某个速率。当网络条件改善时, 客户则随后能够选择来自较高比特率的块。

通过动态地监视可用带宽和客户缓存等级, 随着版本切换而调节传输速率, DASH 经常能够在可能的最好质量等级下实现连续播放, 而没有帧停滞或跳跃。此外, 由于客户 (而不是服务器) 掌控相关信息以决定下一次发送哪个块, 所以这个方案也改善了服务器侧的扩展性。该方法的另一个好处是客户能够使用 HTTP 字节范围请求来精确地控制预取的视频量, 这些视频将缓存在客户本地。

在提及许多实现后, 我们对 DASH 的简要讨论进行总结, 服务器不仅存储视频的许多版本, 还单独存储了音频的许多版本。每个音频版本有自己的质量等级和比特率以及自己的 URL。在这些实现中, 客户动态地选择视频和音频块, 本地同步音频和视频播放。

7.2.4 内容分发网

今天, 许多因特网视频公司日复一日地向数以百万计的用户按需分发每秒数兆比特的流。例如, YouTube 的视频库藏有几亿个, 每天向全世界的用户分发几亿条流 [Ding 2011]。向位于全世界的所有用户流式传输所有流量同时提供连续播放和高交互性显然是一项有挑战性的任务。

对于一个因特网视频公司，或许提供流式视频服务最为直接的方法是建立单一的大规模数据中心，在数据中心中存储其所有视频，并直接从该数据中心向世界范围的客户传输流式视频。但是这种方法存在三个问题。首先，如果客户远离数据中心，服务器到客户的分组将跨越许多通信链路并很可能通过许多 ISP，其中某些 ISP 可能位于不同的大洲。如果这些链路之一提供的吞吐量小于视频消耗速率，端到端吞吐量也将小于该消耗速率，给用户带来恼人的停滞时延。（第 1 章讲过，一条流的端到端吞吐量由瓶颈链路的吞吐量所决定。）出现这种事件的可能性随着端到端路径中链路数量的增加而增加。第二个缺陷是流行的视频很可能经过相同的通信链路发送许多次。这不仅浪费了网络带宽，因特网视频公司自己也将为向因特网反复发送相同的字节而向其 ISP 运营商（连接到数据中心）支付费用。这种解决方案的第三个问题是单个数据中心代表一个单点故障，如果数据中心或其通向因特网的链路崩溃，它将不能够分发任何视频流了。

为了应对向分布于全世界的用户分发巨量视频数据的挑战，几乎所有主要的视频流公司都利用内容分发网（Content Distribution Network, CDN）。CDN 管理分布在多个地理位置上的服务器，在它的服务器中存储视频（和其他类型的 Web 内容，包括文档、图片和音频）的副本，并且所有试图将每个用户请求定向到一个将提供最好的用户体验的 CDN 位置。CDN 可以是专用 CDN（private CDN），即它由内容提供商自己所拥有；例如，谷歌的 CDN 分发 YouTube 视频和其他类型的内容。另一种 CDN 可以是第三方 CDN（third-party CDN），它代表多个内容提供商分发内容；例如，Akamia 的 CDN 是一个第三方 CDN，在其他用户中分发 Netflix 和 Hulu。现代 CDN 的一个可读性强的展望见 [Leighton 2009]。

学习案例

谷歌的网络基础设施

为了支持谷歌的巨量云服务阵列，包括搜索、gmail、日程表、YouTube 视频、地图、文档和社交网络，谷歌已经部署了一个广泛的专用网和 CDN 基础设施。谷歌的 CDN 基础设施具有三个等级的服务器集群：

- 8 个“大型数据中心”，其中 6 个位于美国，2 个位于欧洲 [Google Locations 2012]，每个数据中心具有 10 万台左右的服务器。这些“大型数据中心”负责服务于动态的（并且经常是个性化的）内容，包括搜索结果和 gmail 报文。
- 大约 30 个“邀请做客（bring-home）服务器”集群（参见 7.2.4 节），其中每个集群大约由 100 ~ 500 台服务器组成 [Adhikari 2011a]。这些集群位置分布于全球，每个位置通常靠近多个第一层 ISP PoP。这些集群负责服务于静态内容，包括 YouTube 视频 [Adhikari 2011a]。
- 数以百计的“深入（enter-deep）服务器”集群（参见 7.2.4 节），每个集群位于一个接入 ISP 中。这里一个集群通常由位于一个机架上的数十台服务器组成。这些“深入服务器”执行 TCP 分岔（参见 3.7 节）并服务于静态内容 [Chen 2011]，包括体现搜索结果的 Web 网页的静态部分。

所有这些数据中心和集群位置与谷歌自己的专用网连接在一起，作为一个巨大的 AS（AS 15169）的一部分。当某用户进行搜索请求时，该请求常常先经过本地 ISP 发送

到邻近的“深入服务器”缓存中，从这里检索静态内容；同时将该静态内容提供给客户，邻近的缓存也经谷歌的专用网将请求转发给“大型数据中心”，从这里检索个性化的搜索结果。对于某 YouTube 视频，该视频本身可能来自一个“邀请坐客服务器”缓存，而围绕该视频的 Web 网页部分可能来自邻近的“深入服务器”缓存，围绕该视频的广告来自数据中心。总的来说，除了本地 ISP，谷歌云服务在很大程度上是由独立于公共因特网的网络基础设施提供的。

CDN 通常采用两种不同的服务器安置原则 [Huang 2008]：

- **深入。**第一个原则由 Akamai 首创，该原则是通过在遍及全球的接入 ISP 中部署服务器集群来深入到 ISP 的接入网中。（在 1.3 节中描述了接入网。）Akamai 在大约 1700 个位置采用这种方法部署集群。其目标是靠近端用户，通过减少端用户和 CDN 集群之间（内容从这里收到）链路和路由器的数量，从而改善了用户感受的时延和吞吐量。因为这种高度分布式设计，维护和管理集群的任务成为挑战。
- **邀请做客。**第二个设计原则由 Limelight 和许多其他 CDN 公司所采用，该原则是通过在少量（例如 10 个）关键位置建造大集群并使用专用高速网络连接这些集群来邀请到 ISP 做客。不是将集群放在接入 ISP 中，这些 CDN 通常将每个集群放置在这样的位置上，即同时靠近许多第一层 ISP 的 PoP 的位置（参见 1.3 节），例如，一个大城市中同时靠近 AT&T PoP 和 Verizon PoP 的几英里范围内。与深入设计原则相比，邀请做客设计通常产生较低的维护和管理开销，可能以对端用户的较高时延和较低吞吐量为代价。

一旦 CDN 的集群准备就绪，它就可以跨集群复制内容。CDN 可能不希望将每个视频的副本放置在每个集群中，因为某些视频很少观看或仅在某些国家中流行，事实上，许多 CDN 没有将视频推入它们的集群，而是使用一种简单的拉策略：如果客户向一个未存储该视频的集群请求某视频，则该集群检索该视频（从某中心仓库或者从另一个集群），向客户流式传输视频时的同时在本地存储一个副本。类似于因特网缓存（参见第 2 章），当某集群存储器变满时，它删除不经常请求的视频。

1. CDN 操作

在讨论过这两种部署 CDN 的重要方法后，我们现在深入看看 CDN 操作的细节。当用户主机中的一个浏览器指令检索一个特定的视频（由 URL 标识）时，CDN 必须截获该请求，以便能够：①确定此时适合用于该客户的 CDN 服务器集群；②将客户的请求重定向到该集群的某台服务器。我们很快将讨论 CDN 是如何能够确定一个适当的集群的。但是我们首先考察截获和重定向请求所依赖的机制。

大多数 CDN 利用 DNS 来截获和重定向请求；这种使用 DNS 的一个有趣讨论见 [Vixie 2009]。我们考虑用一个简单的例子来说明通常是怎样涉及 DNS 的。假定一个内容提供商 NetCinema，雇佣了第三方 CDN 公司 KingCDN 来向其客户分发视频。在 NetCinema 的 Web 网页上，它的每个视频都被指派了一个 URL，该 URL 包括了字符串“video”以及该视频本身的独特标识符；例如，转换器 7 可以指派为 `http://video.netcinema.com/6Y7B23V`。接下来出现如图 7-4 所示的 6 个步骤：

- 1) 用户访问位于 NetCinema 的 Web 网页。

2) 当用户点击链接 `http://video.netcinema.com/6Y7B23V` 时, 该用户主机发送了一个对于 `video.netcinema.com` 的 DNS 请求。

3) 用户的本地 DNS 服务器 (LDNS) 中继该 DNS 请求到一台用于 NetCinema 的权威 DNS 服务器, 该服务器观察到主机名 `video.netcinema.com` 中的字符串 “video”。为了将该 DNS 请求移交给 KingCDN, NetCinema 权威 DNS 服务器并不返回一个 IP 地址, 而是向 LDNS 返回一个 KingCDN 域的主机名, 如 `a1105.kingcdn.com`。

4) 从这时起, DNS 请求进入了 KingCDN 专用 DNS 基础设施。用户的 LDNS 则发送第二个请求, 此时是对 `a1105.kingcdn.com` 的 DNS 请求, KingCDN 的 DNS 系统最终向 LDNS 返回 KingCDN 内容服务器的 IP 地址。所以正是在这里, 在 KingCDN 的 DNS 系统中, 指定了 CDN 服务器, 客户将能够从这台服务器接收到它的内容。

5) LDNS 向用户主机转发内容服务 CDN 结点的 IP 地址。

6) 一旦客户收到 KingCDN 内容服务器的 IP 地址, 它与具有该 IP 地址的服务器创建了一条直接的 TCP 连接, 并且发出对该视频的 HTTP GET 请求。如果使用了 DASH, 服务器将首先向客户发送具有 URL 列表的告示文件, 每个 URL 对应视频的每个版本, 并且客户将动态地选择来自不同版本的块。

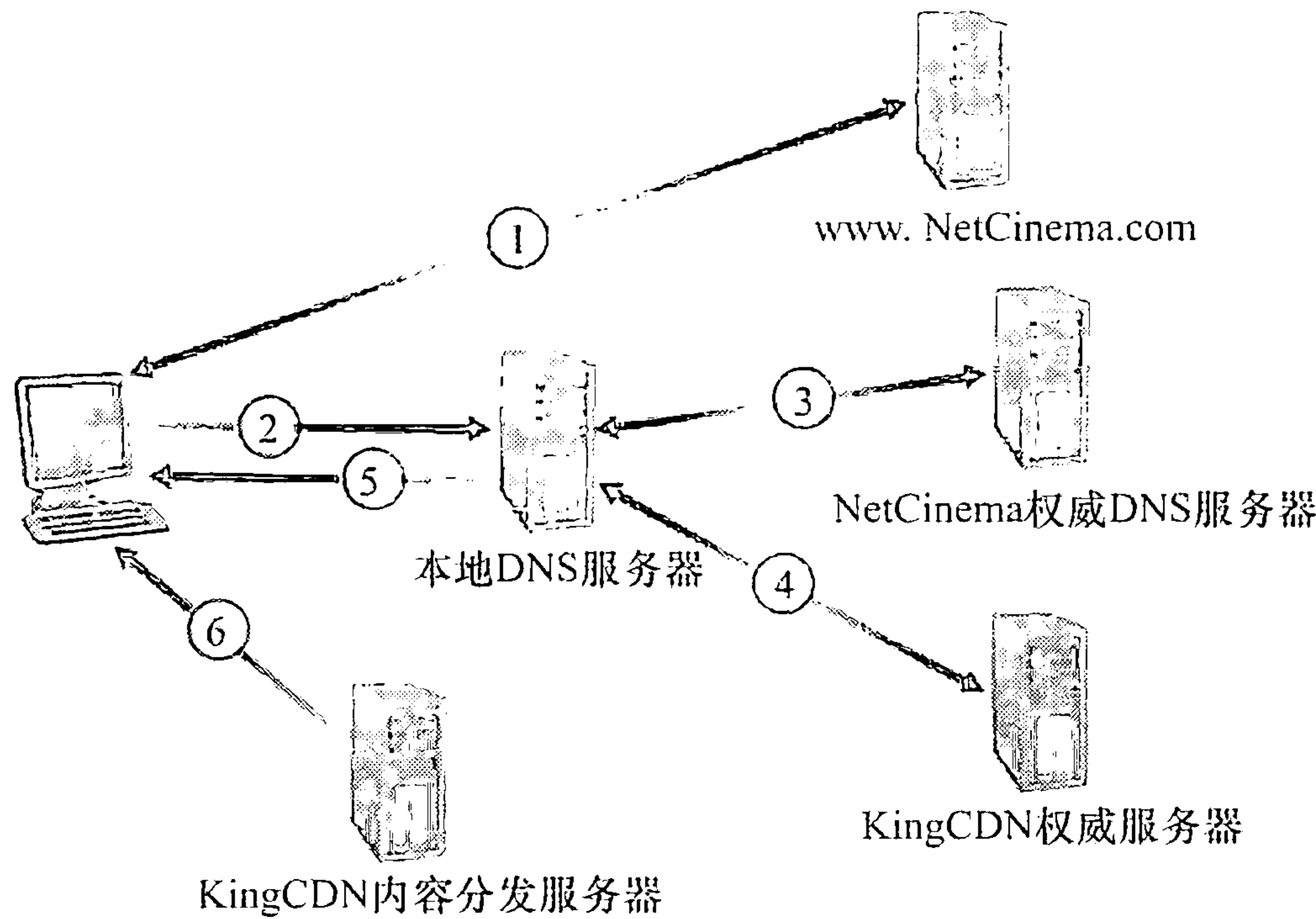


图 7-4 DNS 将用户的请求重定向到一台 CDN 服务器

2. 集群选择策略

任何 CDN 部署, 其核心是**集群选择策略** (cluster selection strategy), 即动态地将客户定向到 CDN 中服务器集群或数据中心的机制。如我们刚才所见, CDN 经过客户的 DNS 查找得知了该客户的 LDNS 服务器的 IP 地址。在得知该 IP 地址之后, CDN 需要基于该 IP 地址选择一个适当的集群。CDN 一般采用专用的集群选择策略。我们现在简单地介绍一些自然的策略, 每种策略都有其优点和缺点。

一种简单的策略是指派客户到地理上最为邻近 (geographically closest) 的集群。使用商用地理位置数据库 (例如 Quova [Quova 2012] 和 Max-Mind [MaxMind 2012]), 每个 LDNS IP 地址都映射到一个地理位置。当从一个特殊的 LDNS 接收到一个 DNS 请求时, CDN 选择地理上最为接近的集群, 即离 LDNS 几千米远的集群, “就像鸟飞一样”。这样的解决方案对于众多用户来说能够工作得相当好 [Agarwal 2009]。但对于某些客户, 该解决

方案可能执行的效果差，因为地理最邻近的集群可能并不是沿着网络路径最近的集群。此外，一种所有基于 DNS 的方法都内在具有的问题是某些端用户配置使用位于远地的 LDNS [Shaikh 2001; Mao 2002]，在这种情况下，LDNS 位置可能远离客户的位置。此外，这种简单的策略忽略了时延和可用带宽随因特网路径时间而变化，总是为特定的客户指派相同的集群。

为了基于当前流量条件为客户决定最好的集群，CDN 能够对其集群和客户之间的时延和丢包性能执行周期性的**实时测量**（real-time measurement）。例如，CDN 能够让它的每个集群周期性地向位于全世界的所有 LDNS 发送探测分组（例如，ping 报文或 DNS 请求）。这种方法的一个缺点是许多 LDNS 配置为不会对这些探测进行响应。

发送测量路径性质的外部流量的另一种方法是，使用客户与 CDN 服务器之间近期和进行中的流量特点。例如，客户与集群之间的时延能够通过观察 TCP 三次握手过程中服务器到客户 SYNACK 和客户到服务器 ACK 之间的间隙进行估计。然而，这种解决方案为了测量到这些集群路径的性质，时不时地需要将客户重定向到（可能的）次优化集群。尽管仅有少量的请求需要作为探测分组而工作，但选择的客户在接收内容（视频或其他）时可能经受很大的性能下降 [Andrews 2002; Krishan 2009]。对于集群到客户路径的另一种探测方法是使用 DNS 请求流量来测量客户和集群之间的时延。具体而言，在 DNS 阶段（在图 7-4 步骤 4 中），客户的 LDNS 能够偶尔地定向到不同的权威 DNS 服务器，这些服务器可以安装在各个集群位置，产生 DNS 流量，这些流量则能够在 LDNS 和这些集群位置之间测量到。使用这种方案，DNS 服务器继续向该客户返回优化的集群，使得交付视频和其他 Web 对象不会受到损害 [Huang 2010]。

使客户与 CDN 服务器匹配的一种非常不同的方法是使用 IP 任播（IP anycast）[RFC 1546]。IP 任播基于的思想是让因特网中的路由器将客户的分组路由到“最近的”集群，就像由 BGP 决定的那样。具体而言，如图 7-5 中所示，在 IP 任播配置阶段，CDN 公司为它的每个集群指派相同的 IP 地址，并且使用标准的 BGP 从每个不同的集群位置来通告该 IP 地址。当一个 BGP 路由器接收到对这个相同的 IP 地址的多个路由通告时，它对待这些通告就像对相同的物理位置提供了不同的路径（事实上，在这个时候，这些通告对不同的物理位置用了不同的路径）。遵循标准的操作过程，则 BGP 路由器将根据其本地路由选择机制，对该 IP 地址选择“最好的”（例如最近的，就像 AS 跳计数所决定的那样）路由。例如，如果一个 BGP 路由（对应于一个位置）离该路由器仅一个 AS 跳的距离，并且所有其他 BGP 路由（对应于其他位置）是两个或更多 AS 跳，则 BGP 路由器将通常选择使分组路由到需要仅通过一个 AS 的位置（参见 4.6 节）。在这个初始配置阶段后，CDN 能够从事内容分发的主要工作。当任何客户要观看任何视频时，CDN 的 DNS 返回该任播地址，而无论该客户位于何处。当客户向该 IP 地址发送分组时，该分组被路由到“最近的”集群，如同由预先配置的转发表决定的那样，该转发表用 BGP 进行配置，如刚才描述的那样。这种方法具有如下优点，发现的集群最靠近客户而不是最靠近该客户的 LDNS。然而，IP 任播策略仍未顾及因特网在短时间范围内的动态性质 [Ballani 2006]。

除了诸如时延、丢包和带宽性能等网络相关考虑外，在设计集群选择策略时还有许多要考虑的重要因素。集群上的负载就是一个这样的因素，即客户不应当定向到过载的集群。ISP 交付成本是另外一个因素，即可以选择特定的集群，使得用特定的 ISP 承载 CDN 到客户的流量，兼顾到 ISP 和集群操作者之间的契约关系中的不同的成本结构。

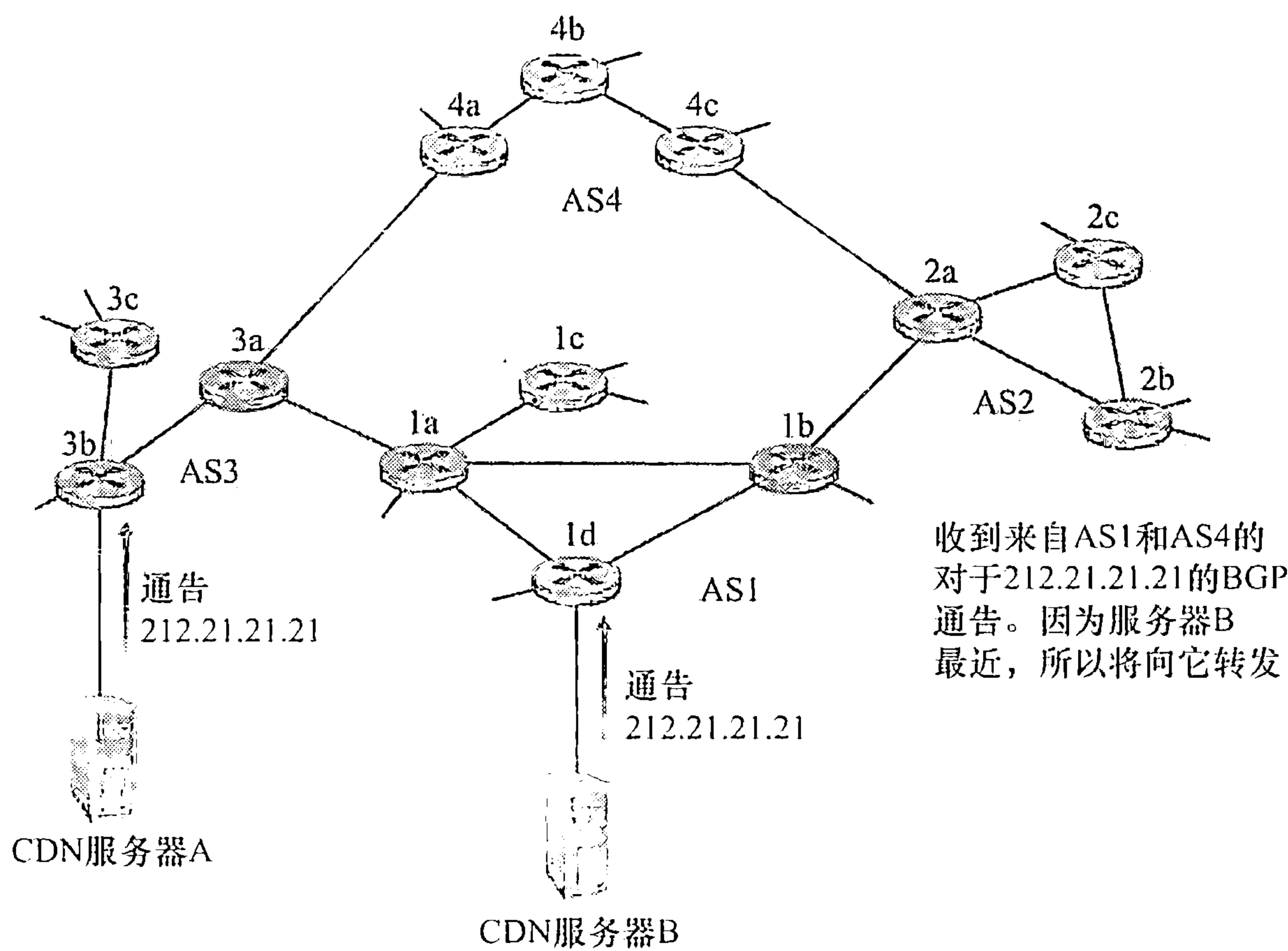


图 7-5 使用 IP 任播将客户路由到最近的 CDN 集群

7.2.5 学习案例：Netflix、YouTube 和 “看看”

通过观察三个高度成功的大规模公司：Netflix、YouTube 和 “看看”，我们来总结流式存储视频的讨论。我们将看到，这些系统采用的方法差异很大，但却应用了在本节中讨论的许多支撑原则。

1. Netflix

Netflix 在 2011 年产生了几乎 30% 的美国因特网下载流量，它已经成为美国首屈一指的在线电影和 TV 节目的服务提供商 [Sandvine 2011]。为了迅速地部署它的大规模服务，Netflix 已经广泛地使用了第三方的云服务和 CDN。Netflix 的确是部署大规模在线服务的有趣典范，它通过从第三方租用服务器、带宽、存储和数据库服务而几乎不使用任何自己的基础设施。下列讨论节选自可读性很强的 Netflix 体系结构测量研究 [Adhikari 2012]。如我们将看到的那样，Netflix 应用了本节前面涉及的许多技术，包括使用一个 CDN（实际上是多个 CDN）的视频分发和经 HTTP 的适应性流。

图 7-6 显示了 Netflix 视频流平台的基本体系结构。它具有四个主要构件：注册和支付服务器、亚马逊云、多个 CDN 提供商和客户。在它自己的硬件基础设施中，Netflix 维护注册和支付服务器，这些服务器处理新账户的注册和捕获信用卡支付信息。除了这些基本的功能外，Netflix 通过利用亚马逊云中的机器（或虚拟机）来运行它的在线服务。在亚马逊云中发生的某些功能包括：

- 内容摄取。在 Netflix 能够向它的用户分发某电影之前，它必须首先摄取和处理该电影。Netflix 接收电影的制片厂母带，并且将其上载到亚马逊云中的主机上。
- 内容处理。亚马逊云中的机器为每部电影生成许多不同格式，适合于不同类型的客户视频播放器在桌面计算机、智能手机和与电视机相连的游戏机上运行。为每种不同格式和比特率的视频都生成一种不同的版本，允许使用 DASH 经 HTTP 适应性播放流。

- 向 CDN 上载版本。一旦某电影的所有版本均已生成，在亚马逊云中的主机将向 CDN 上载这些版本。

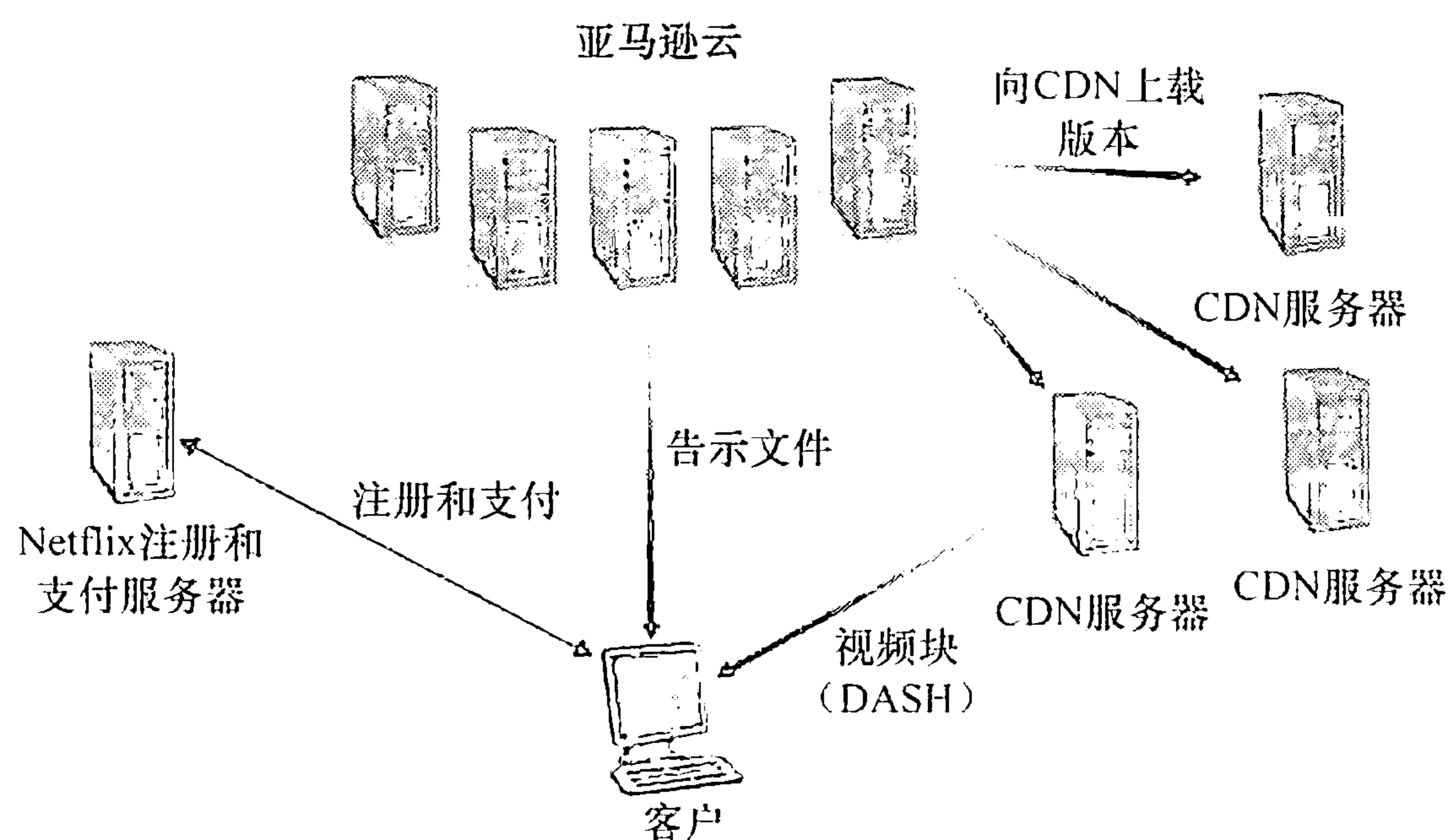


图 7-6 Netflix 视频流平台

为了向其用户按需交付电影，Netflix 广泛地使用了 CDN 技术。事实上，在 2012 年本书写作时，Netflix 没有应用一个自己的 CDN，但却同时使用了 3 个第三方 CDN 公司，即 Akamai、Limelight 和 Level-3。

描述了 Netflix 体系结构的组件后，我们来仔细观察一下客户与各台服务器之间的交互，这些服务器与电影交付有关。浏览 Netflix 视频库的 Web 网页由来自亚马逊云中的服务器提供服务。当用户选择一个电影准备“现在播放”，该用户也从亚马逊云中的服务器获得了一个告示文件。该告示文件提供了各种各样的信息，包括 CDN 的等级列表和对于不同版本电影的 URL，这些都被用于 DASH 重放。CDN 的等级由 Netflix 决定，并且可能随流式会话的改变而变化。客户通常将在告示文件中选择等级最高的 CDN。在客户选择一个 CDN 后，CDN 促使 DNS 将客户重定向到一个特定的 CDN 服务器，如 7.2.4 节所述。该客户和那台 CDN 服务器则使用 DASH 进行交互。具体而言，如 7.2.3 节所述，客户使用 HTTP GET 请求报文中的字节范围首部，以请求来自电影的不同版本的块。Netflix 使用大约 4 秒长的块 [Adhikari 2012]。随着这些块的下载，客户测量收到的吞吐量并且运行一个速率确定算法来确定下一个请求块的质量。

Netflix 包含了本节前面讨论的许多关键原则，包括适应性流和 CDN 分布。Netflix 也很好地区分了一个产生几乎 30% 因特网流量的主要因特网服务能够几乎完全在第三方云和第三方 CDN 基础设施上运行，却几乎不必使用自己的基础设施的情况！

2. YouTube

在 YouTube 的库中有大约 5 亿个视频，并且每天有 5 亿次视频观看 [Ding 2011]，毫无疑问 YouTube 是世界上最大的视频共享站点。YouTube 于 2005 年 4 月开始它的服务，并于 2006 年 11 月被谷歌公司收购。尽管谷歌/YouTube 设计和协议是专用的，但通过几项独立的测量结果，我们能够基本理解 YouTube 的工作原理 [Zink 2009; Torres 2011; Adhikari 2011a]。

与 Netflix 一样，YouTube 广泛地利用 CDN 技术来分发它的视频 [Torres 2011]。然而，

与 Netflix 不同的是，谷歌不应用第三方 CDN，而是使用自己的专用 CDN 来分发 YouTube 视频。谷歌在数百个不同的位置安装了服务器集群，从这些位置的子集（大约有 50 个）分发 YouTube 视频 [Adhikari 2011a]。谷歌使用 DNS 将用户请求重定向到一个特定的集群，如 7.2.4 节所述。在大部分时间中，谷歌的集群选择策略将客户定向到使客户与集群之间的 RTT 最低的集群；然而，为了平衡跨集群的负载，有时客户也被定向（经 DNS）到一个更远的集群 [Torres 2011]。此外，如果一个集群不具有请求的视频，该集群并不是从某个别的地方获取该视频并中继给客户，而是可能返回一个 HTTP 重定向报文，从而将该客户重定向到另一个集群 [Torres 2011]。

YouTube 应用 HTTP 流，如 7.2.2 节中所讨论的。YouTube 经常使少量的不同版本为一个视频可用，每个具有不同的比特率和对应的质量等级。到了 2011 年，YouTube 没有应用适应性流（例如 DASH），而要求用户人工选择一个版本。为了节省那些将被重定位或提前终止而浪费的带宽和服务器资源，YouTube 在获取视频的目标量之后，使用 HTTP 字节范围请求来限制传输的数据流。

每天有几百万视频被上载到 YouTube。不仅 YouTube 视频经 HTTP 以流方式从服务器到客户，而且 YouTube 上载者也经 HTTP 从客户到服务器上载他们的视频。YouTube 处理它收到的每个视频，将它转换为 YouTube 视频格式并且创建具有不同比特率的多个版本。这种处理完全发生在谷歌数据中心。因此，宏观地对比 Netflix，Netflix 几乎完全在第三方基础设施上运行，而谷歌在它自己巨大的基础设施上运行整个 YouTube 服务，该基础设施是由数据中心、专用 CDN 和互联其数据中心的专用全球网络和 CDN 集群构成的。（参见 7.2.4 节中有关谷歌的网络基础设施的学习案例。）

3. “看看”

我们刚才看过了 Netflix 和 YouTube 服务，由 CDN（或者第三方或者专用 CDN）运行的服务器以流的方式把视频传送给客户。Netflix 和 YouTube 不仅必须为服务器硬件付费（通过自己拥有直接付费，或者通过租用间接付费），而且要为服务器分发视频时使用的带宽付费。鉴于这些服务的规模和它们消耗的带宽量，这种“客户-服务器”的部署代价是极高的。

我们通过描述一种完全不同的方法来总结本节内容，即大规模经因特网提供按需视频。这是一种允许服务提供商极大地减少其基础设施和带宽成本的方法。如你可能猜测的那样，这种方法使用 P2P 交付，而不是客户-服务器（经 CDN）交付。几个中国的公司非常成功地使用了 P2P 视频交付，包括“看看”（由迅雷公司拥有并运行）、PPTV（以前为 PPLive）和“皮皮”（以前为 PPstream）。“看看”当前是首屈一指的基于 P2P 按需视频的中国提供商，每月有超过 2000 万不同的用户观看它的视频。

在一个高层面看，P2P 视频流非常类似于 BitTorrent 附件下载（在第 2 章讨论过）。当一个对等方要看一个视频时，它联系一个跟踪器（它可能是集中式的，或者是使用 DHT 的基于对等方的），以发现在系统中具有该视频副本的其他对等方。这个对等方则并行地从具有该文件的这些其他对等方请求该视频文件中的块。然而，不同于使用 BitTorrent 下载，请求被优先地给予那些即将播放的块，以确保连续的播放。

“看看”设计应用了一个跟踪器和它自己的 DHT 来跟踪内容。对于最流行内容的蜂群规模涉及数万对等方，通常大于 BitTorrent 中的最大蜂群 [Dhungel 2012]。“看看”的协

议都是专用的，这些协议用于对等方和跟踪器之间、对等方和 DHT 之间和对等方之间的通信。有趣的是，为了在对等方之间分发视频块，“看看”每当可能时都使用 UDP，这导致在中国的因特网中有巨量的 UDP 流量 [Zhang M 2010]。

7.3 IP 语音

经因特网的实时会话式语音经常被称为因特网电话 (Internet telephony)，因为从用户的视角看，它类似于传统的电路交换电话服务。它通常也被称为 IP 语音 (Voice-over-IP, VoIP)。在本节中我们描述 VoIP 所依据的原则和协议。会话式视频在许多方面类似于 VoIP，除了它包括参与者的视频以及他们的语音以外。为了使讨论重点突出且具体，我们这里仅关注语音，而不是语音和视频的结合。

7.3.1 尽力而为服务的限制

因特网的网络层协议 IP 提供了尽力而为的服务。那就是说服务尽全力将每个数据报从源尽可能快地移动到目的地，但是没有就在某些时延界限内使分组到达目的地或丢包百分比的限制做任何承诺。缺失这种保证对实时会话式应用的设计提出了严峻挑战，这些应用对分组时延、时延抖动和丢包非常敏感。

在本节中，我们将讨论加强尽力而为网络上的 VoIP 性能的几种方式。我们的重点将在应用层技术上，即这些技术并不要求在网络核心甚至在端系统的运输层有任何变化。为了使讨论具体，我们将讨论在一个特定的 VoIP 例子环境下尽力而为 IP 服务的限制。发送方以每秒 8000 字节的速率产生字节，且每 20ms 将字节汇聚成块。每个块和一个特殊的首部（在下面讨论）封装在一个 UDP 报文段中（通过一个到套接字接口的呼叫）。因此，一个块中的字节数为 $(20\text{ms}) \times (8000 \text{ 字节/秒}) = 160 \text{ 字节}$ ，每 20ms 发送一个 UDP 报文段。

如果每个分组以恒定的端到端时延到达接收方，那么分组每隔 20ms 就能周期性地到达接收方。在这种理想的情况下，只要每个块一到达，接收方就能直接播放它。但不幸的是，某些分组可能丢失，大多数分组没有相同的端到端时延，即使在一个轻度拥塞的因特网中也是如此。因此，接收方必须更仔细地判断：①什么时候播放一个块；②如何处理一个丢失块。

1. 丢包

考虑由 VoIP 应用产生的一个 UDP 报文段。这个 UDP 报文段封装在 IP 数据报中。当数据报在网络中徘徊时，在等待出链路传输时它要经过路由器的缓存（即队列）。从发送方到接收方的路径上的一个或多个缓存有可能是满的，不能接纳该 IP 数据报。在这种情况下，这个 IP 数据报就被丢弃了，永远不会到达接收方的应用程序。

通过 TCP（它提供了可靠数据传输）而不是 UDP 发送分组可以消除丢失。然而，重传机制对于诸如 VoIP 这样的会话式实时音频应用，通常认为是不可接受的，因为它们增加了端到端时延 [Bolot 1996]。此外，当丢包后，由于 TCP 的拥塞控制，发送方的传输速率可能减少到低于接收方的排空速率，可能导致缓存“饥饿”。这可能会对接收方的语音可理解程度产生严重影响。由于这些原因，几乎所有现有的 VoIP 应用默认运行在 UDP 上。[Baset 2006] 报告称 Skype 使用了 UDP，除非用户位于阻碍 UDP 报文段的 NAT 或防

防火墙之后（这时使用 TCP）。

但是分组的丢失并不一定会造成人们想象中的灾难。实际上，取决于语音是如何编码和传输的以及接收方隐藏丢包的方式，1% ~ 20% 的丢包率是可以忍受的。例如，前向纠错（FEC）能够有助于隐藏丢包。我们后面可以看到，通过使用 FEC，将冗余信息和初始信息一起传输，以便能够从冗余信息中恢复一些丢失的初始数据。无论如何，如果发送方和接收方之间的一段或多段链路严重拥塞，丢包率超过 10% ~ 20%（例如在无线链路上），那么无论采取何种措施都无法获得可以接受的声音质量了。显然，尽力而为服务有它的局限性。

2. 端到端时延

端到端时延（end-to-end delay）是以下因素的总和：路由器中的传输、处理和排队时延，链路中的传播时延和端系统的处理时延。对于实时会话式应用，例如 VoIP，听电话的人对于小于 150ms 的端到端时延是觉察不到的；在 150ms 和 400ms 之间的时延能够接受，但是不够理想；超过 400ms 的时延可能严重妨碍语音谈话的交互性。VoIP 应用程序的接收方通常忽略时延超过特定阈值（例如超过 400ms）的任何分组。因此，时延超过该阈值的分组等效于丢弃。

3. 分组时延抖动

端到端时延的一个关键成分是一个分组在网络路由器中经历的变化的排队时延。由于这些可变的时延，从在源中产生分组到它在接收方收到的这段时间，对于不同的分组可能会有波动，如图 7-1 所示。这个现象称为时延抖动（jitter）。举一个例子，考虑在 VoIP 应用中两个连续的分组。发送方在发送第一个分组 20ms 之后发送第二个分组。但是在接收方，这两个分组之间的间隔可能变得大于 20ms。为了理解这一点，假设第一个分组到达路由器的一个几乎为空的队列，但是恰好在第二个分组到达该队列之前，从其他源来的大量分组也到达了相同的队列。因为在这个路由器中第一个分组经受了很小的排队时延，而第二个分组经受了较大的排队时延，这两个连续的分组之间的时间间隔变得大于 20ms 了。两个连续分组的间隔也可能会小于 20ms。为了理解这种情况，再次考虑两个连续的分组。假设第一个分组加入一个有大量分组队列的队尾，并且第二个分组到达这个队列时第一个分组尚未被传输，而且来自其他源的分组尚未到达该队列。在这种情况下，这两个分组发现它们在队列中互相紧挨着。如果在路由器的出链路传输一个分组所需时间小于 20ms，则第一个分组和第二个分组的间隔就变得小于 20ms 了。

这种情况可以与在公路上开车相类比。假设你和你的朋友每人驾驶一辆车从圣地亚哥到凤凰城。并且假设你和朋友有类似的驾驶风格，而且你们都以交通规则允许的 100km/h 的速度驾驶。如果你的朋友在你之前一个小时出发，那么根据干扰车流量的不同，你可能在你的朋友之后一个小时左右到达凤凰城。

如果接收方忽略了时延抖动的存在，一旦该块到达就开始播放，那么在接收方产生的音频质量很容易变得不可理解。幸运的是，时延抖动通常可以通过使用序号（sequence number）、时间戳（timestamp）和播放时延（playout delay）来消除，如下面所讨论的内容。

7.3.2 在接收方消除音频的时延抖动

对于 VoIP 应用，周期性地产生分组，接收方应该在存在随机网络时延抖动的情况下

尝试提供播放语音块。这经常通过结合下面两种机制来实现：

- 为每个块预先计划一个时间戳（timestamp）。发送方用每个块产生的时刻为它加上时间印记。
- 在接收方延迟播放（delaying playout）块。如我们前面在图 7-1 的讨论所见，接收的音频块的播放时延必须足够长，以便大多数分组在它们的预定播放时间之前被接收到。这个播放时延可能在整个音频会话期间是固定的，或者在音频会话生命期中适应性地变化。

我们现在讨论如何结合这两种机制来减轻甚至消除时延抖动的影响。我们研究两种播放策略：固定播放时延和适应性播放时延。

1. 固定播放时延

使用固定播放时延策略，接收方试图在块产生正好 q ms 后播放它。因此如果一个块在时刻 t 打上时间戳，接收方在时刻 $t + q$ 播放这个块，假设这个块在那个时间已经到达。在预定播放时间之后到达的分组将被丢弃，并被认为已经丢失。

q 选择什么值为好呢？VoIP 能够支持高达约 400ms 的时延，尽管使用更小的 q 值可以获得更令人满意的会话体验。另一方面，如果 q 比 400ms 小得多，那么由于网络引入的分组时延抖动会使许多分组可能错过了它们的预定播放时间。概括地说，如果端到端时延经常发生大的变化，用一个大的 q 更好；另一方面，如果时延很小并且时延变化也很小，用一个较小的、可能小于 150ms 的 q 更好。

在图 7-7 中说明了播放时延和丢包之间的折中。该图表示了单个话音突峰期的分组产生和播放的时间。考虑了两种不同的初始播放时延。如最左边的阶梯所示，发送方以规则的间隔（比方说每 20ms）产生一组分组。在这个话音突峰期中的第一个分组在时刻 r 被接收到。如该图所示，由于网络时延抖动，后续分组的到达间隔是不均匀的。

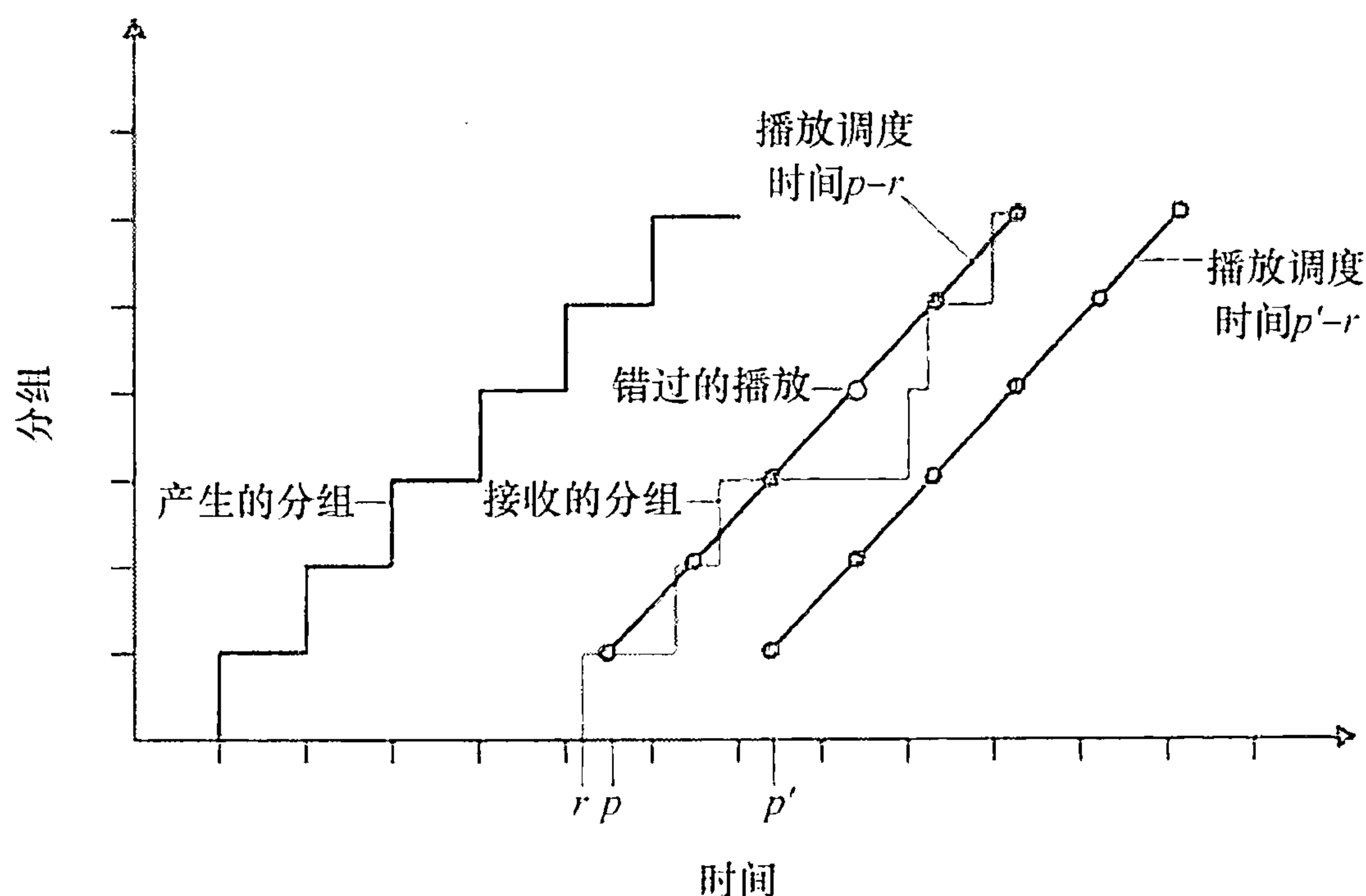


图 7-7 不同的固定播放时延情况下的丢包

对于第一个播放调度时间，固定的初始播放时延设置为 $p - r$ 。使用这个方案，第四个分组没有在它调度的播放时间到达，接收方认为它丢失了。对于第二个调度时间，固定的初始播放时延设置为 $p' - r$ 。对于这个方案，所有分组都在它们调度的播放时间之前到达，因此没有丢失。

2. 适应性播放时延

上面的例子显示了当使用固定播放时延来设计播放策略时所引起的重要的时延与丢包折中。若初始播放时延设置得比较大,大多数分组能在它们的截止时间内到达,因此存在的丢失将可忽略不计;然而,对于如 VoIP 这样的会话式服务,长时延即使不是不能忍受的,至少也是令人厌恶的。在理想情况下,我们希望播放时延最小化,使丢包低于一定百分比的限制。

处理这种折中的自然方法是估计网络时延和网络时延变化,并且在每个话音突峰期的开始相应地调整播放时延。在话音突峰期开始时适应性地调整播放时延将导致发送方静默期的压缩和拉长;然而,静默期的少量压缩和拉长在谈话中是不易觉察的。

根据 [Ramjee 1994],我们现在描述一种接收方可以用于适应性地调整它的播放时延的通用算法。为此,令

t_i = 第 i 个分组的时间戳 = 该分组在发送方产生的时间

r_i = 分组 i 被接收方接收的时间

p_i = 分组 i 在接收方播放的时间

第 i 个分组的端到端网络时延是 $r_i - t_i$ 。由于网络时延抖动,这个时延在不同的分组之间会发生变化。令 d_i 表示接收到第 i 个分组时的平均网络时延的估计值。这个估计值根据如下的时间戳来构造:

$$d_i = (1 - u)d_{i-1} + u(r_i - t_i)$$

式中 u 是一个固定的常数 (例如, $u = 0.01$)。这样 d_i 是观察到的网络时延 $r_1 - t_1, \dots, r_i - t_i$ 的一个平滑均值。这个估计值为最近观察到的网络时延设置了比过去一段时间观察到的网络时延有更大的权重。这种估值的形式不应该是完全陌生的;相似的思想在第 3 章讨论估计 TCP 往返时间时就使用过。令 v_i 表示与估计平均时延的平均时延绝对偏差的估计值。这个估计值也可从这些时间戳构建:

$$v_i = (1 - u)v_{i-1} + u|r_i - t_i - d_i|$$

为每个接收的分组计算估计值 d_i 和 v_i , 尽管它们仅能用于为任何话音突峰期的第一个分组确定播放点。

一旦计算完了这些估计值,接收方为分组播放应用下列的算法。如果分组 i 是一个话音突峰期的第一个分组,它的播放时间 p_i 计算如下:

$$p_i = t_i + d_i + Kv_i$$

这里 K 是一个正的常数 (例如 $K = 4$)。 Kv_i 项的目的是给将来设置足够大的播放时间,以便话音突峰期中只有一小部分到达的分组由于迟到而丢失。在一个话音突峰期中任何后续分组的播放点被计算为对于这个话音突峰期的第一个分组播放时间点的偏移。特别是,令

$$q_i = p_i - t_i$$

表示从话音突峰期的第一个分组产生到它播放的时间长度。如果分组 j 也属于这个话音突峰期,它播放的时刻是

$$p_j = t_j + q_i$$

刚才描述的算法在假设接收方能够辨明一个分组是否是话音突峰期中的第一个分组的情况下很有意义。这能够通过检查每个接收到的分组中的信号能量来做到。

7.3.3 从丢包中恢复

我们已经较为详细地讨论了一个 VoIP 应用能够怎样处理分组时延抖动。我们现在简要地描述在存在丢包的情况下几种试图保护可接受的音频质量的方案。这样的方案被称为**丢包恢复方案** (loss recovery scheme)。这里我们定义了广义的丢包：如果某分组不能到达接收方或者在它调度的播放时间之后才到达，该分组则丢失。我们再次用 VoIP 例子作为描述丢包恢复方案的环境。

如在本节开始提到的那样，在诸如 VoIP 等会话式实时应用中，重传丢失的分组通常是不可行的。的确，重传一个已经错过了播放截止时间的分组是绝对没有意义的。而且重传一个在路由器队列溢出的分组通常不能足够快地完成。由于这些考虑，VoIP 应用通常使用某种类型的丢包预期方案。两种类型的丢包预期方案是**前向纠错** (Forward Error Correction, FEC) 与**交织** (interleaving)。

1. 前向纠错

FEC 的基本思想是给初始的分组流增加冗余信息。以稍微增加传输速率为代价，这些冗余信息可以用来重建一些丢失分组的近似或者准确版本。根据文献 [Bolot 1996] 和 [Perkins 1998]，我们现在概括了两种简单的 FEC 机制。第一种机制是每发送 n 个块之后发送一个冗余编码的块。这个冗余块通过异或 n 个初始块来获得 [Shacham 1990]。以这种方式，在这 $n+1$ 个分组的组中，如果任何一个分组丢失，接收方能够完全重建丢失的分组。但是如果这一组中有两个或更多分组丢失，接收方则无法重建丢失的分组。通过让组的长度 $n+1$ 比较小，当丢失不是很多时，大部分丢失分组都可以恢复。然而组的长度越小，相对增加的传输速率就越大。特别是若传输速率以 $1/n$ 因子增加的话，例如 $n=3$ ，则传输速率增加 33%。此外，这个简单的方案增加了播放时延，因为接收方在能够开始播放之前，必须等待收到整个组的分组。有关 FEC 在多媒体传输上如何工作的更多实践细节参见 [RFC 5109]。

第二个 FEC 机制是发送一个较低分辨率的音频流作为冗余信息。例如，发送方可能创建一个标称的音频流和一个相应的低分辨率、低比特率的音频流。（这个标称流可能是一个 64kbps 的 PCM 编码，而这个较低质量的流可能是一个 13kbps 的 GSM 编码。）这个低比特率流被认为是冗余信息。如图 7-8 所示，发送方通过从这个标称流中取出第 n 个块并附加上第 $(n-1)$ 个块的冗余信息，以构建第 n 个分组。以这种方式，只要没有连续分组的丢失，接收方都可以通过播放和后续分组一起到达的低比特率编码块来隐藏丢失。当然，低比特率块比标称块的质量要低。然而，在一个流主要是由高质量块组成、偶尔出现低质量块并且没有丢失的块的情况下，其整体的音频质量良好。注意到在这种方案中，接收方在播放前只需接收两个分组，因此增加的时延小。此外，如果低比特率编码比标称编码少得多，那么传输速率的额外增加并不大。

为了处理连续丢失，我们能够使用率一个简单的修正方案。发送方不再仅为第 n 个标称块附加上第 $(n-1)$ 个低比特率块，而是附加上第 $(n-1)$ 个和第 $(n-2)$ 个低比特率块，或者附加第 $(n-1)$ 个和第 $(n-3)$ 个低比特率块等等。通过给每个标称块附加上更多低比特率块，在各种恶劣的尽力而为服务环境下接收方的音频质量变得可接受。在另一方面，附加的块增加了传输带宽和播放时延。

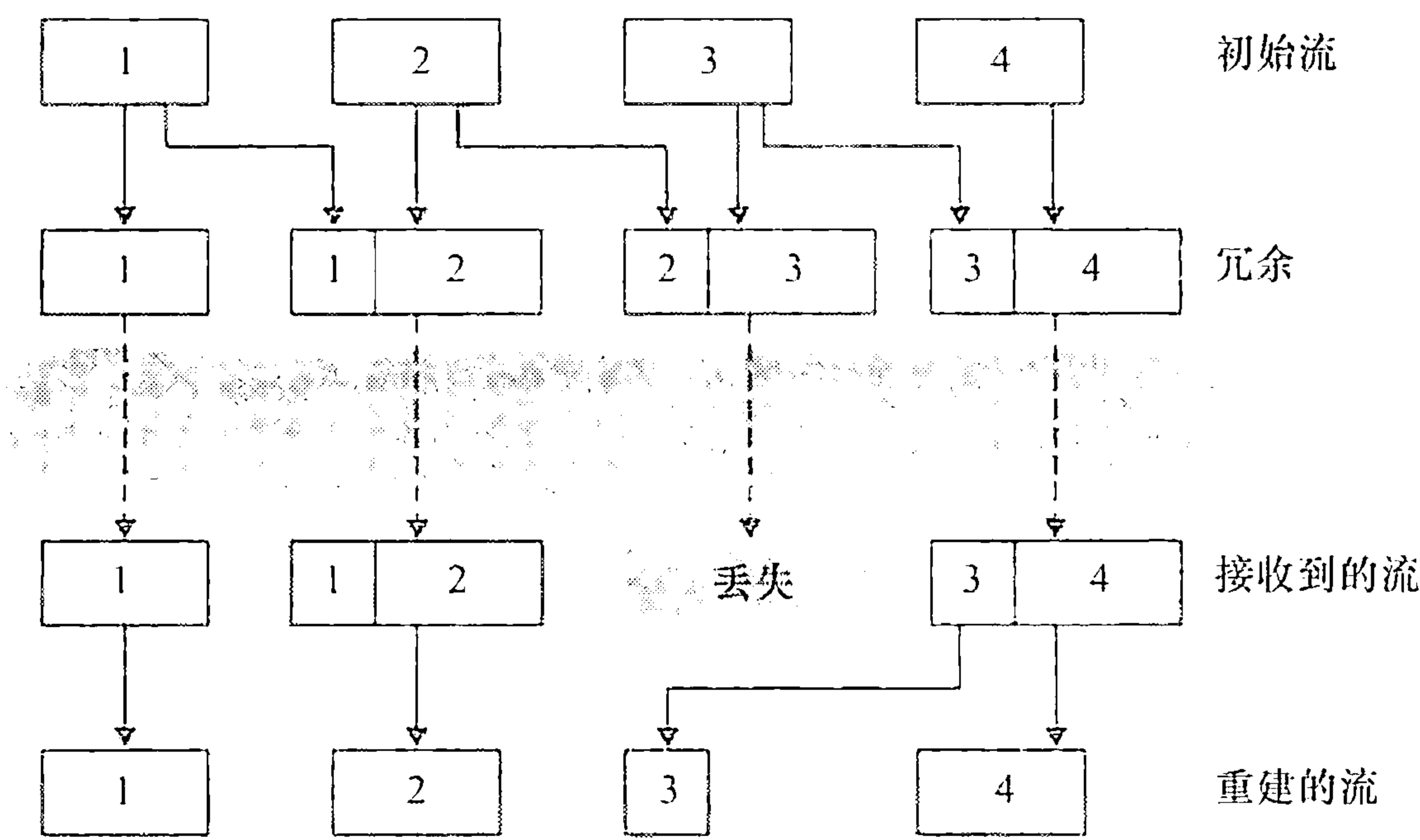


图 7-8 捎带低质量的冗余信息

2. 交织

作为冗余传输的另一种替代方案，VoIP 应用可以发送交织的音频。如图 7-9 所示，发送方在传输之前对音频数据单元重新排序，使得最初相邻的单元在传输流中以一定距离分离开来。交织可以减轻丢包的影响。例如，如果每个单元长为 5ms，块是 20ms（也就是每个块 4 个单元），那么第一个块可能包含 1、5、9 和 13 单元；第二个块可能包含 2、6、10 和 14 单元等等。图 7-9 显示了一个交织流的单个丢包导致重建流中的多个小间隙，这与在非交织流中将会导致单个大间隙形成对照。

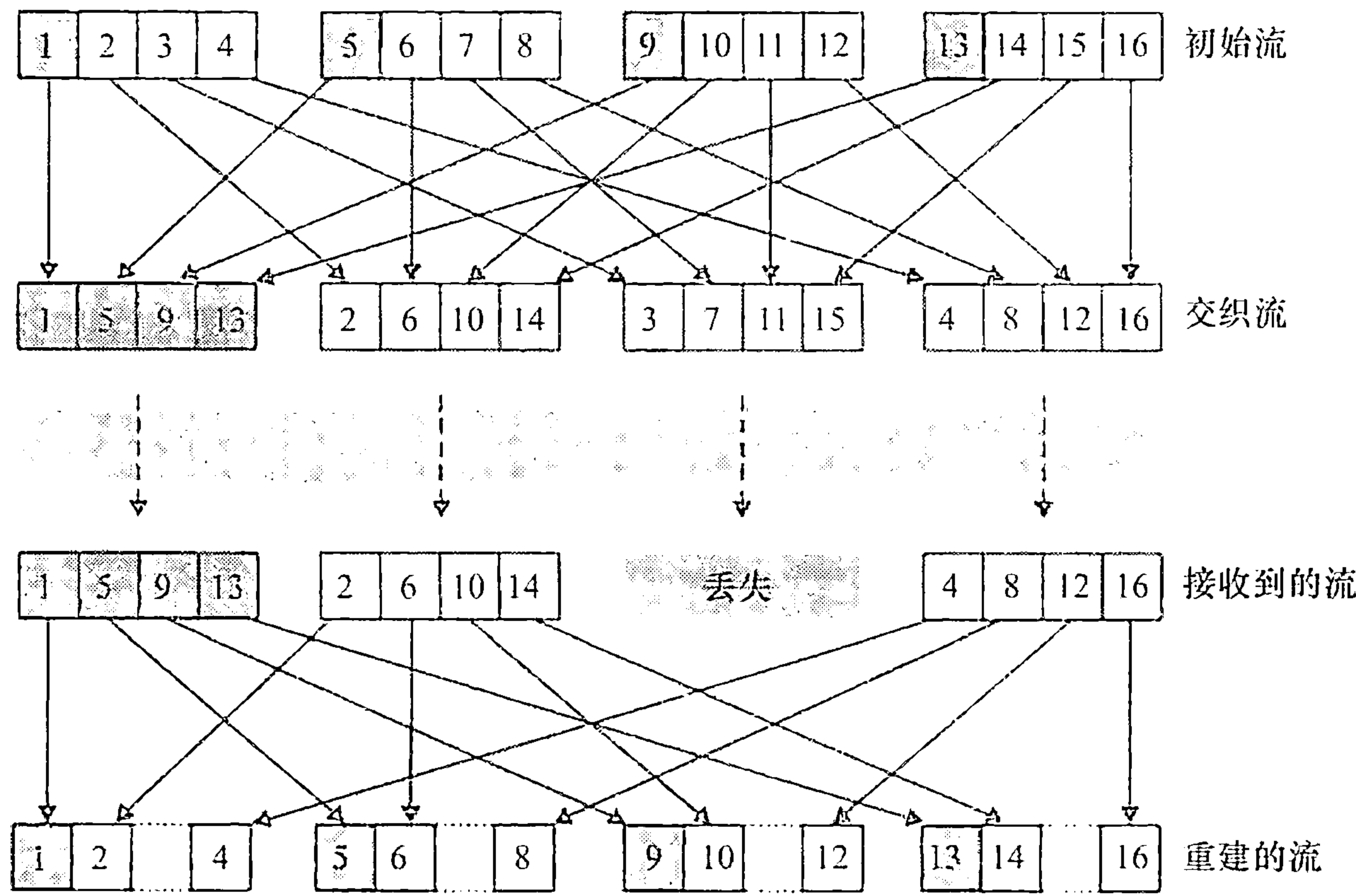


图 7-9 发送交织音频

交织能够明显地提高音频流可感觉到的质量 [Perkins 1998]。它的开销也较低。交织明显的缺点是增加了时延。这限制了它在如 VoIP 这样的会话式应用中的使用，然而它能够很好地处理流式存储音频。交织的一个主要优点是它不增加流的带宽需求。

3. 差错掩盖

差错掩盖方案试图为丢失的分组产生一个与初始分组类似的替代物。正如在 [Perkins

1998] 中讨论的那样, 因为音频信号 (特别是语音) 呈现出大量的短期自相似性, 故该方案是可行的。正因为如此, 这些技术适合于工作在相对小的丢包率 (低于 15%) 和小分组 (4 ~ 40ms) 的情况。当丢失长度接近音素的长度 (5 ~ 100ms) 时, 这些技术就失效了, 因为整个音素可能被听者错过。

也许基于接收方的恢复的最简单方式是分组重复。即用在丢失之前刚到达的分组的副本来代替丢失的分组。这种方法的计算复杂度低, 并且工作得相当好。基于接收方恢复的另一种形式是内插法, 它使用在丢失之前和之后的音频内插形成一个合适分组来隐藏丢失。内插法比分组重复稍微好一些, 但是显然需要更高的计算强度 [Perkins 1998]。

7.3.4 学习案例: 使用 Skype 的 VoIP

Skype 是一个非常流行的 VoIP 应用, 每天都有超过 5000 万个活跃的账户。Skype 除了提供主机到主机的 VoIP 服务, 还提供主机到电话的服务, 电话到主机的服务, 以及多方主机到主机的视频会议服务。(这里, 主机仍然是任一种因特网连接 IP 设备, 包括 PC、平板电脑和智能手机。) Skype 于 2011 年由微软公司出资超过 80 亿美元收购。

因为 Skype 协议是专用的, 并且因为所有 Skype 的控制和媒体分组是加密的, 所以精确地确定 Skype 的工作过程是非常困难的。无论如何, 从 Skype 的 Web 网站和几项测量研究, 研究人员已经知道了 Skype 总体上是怎样工作的 [Baset 2006; Guha 2006; Chen 2006; Suh 2006; Ren 2006; Zhang X 2012]。对于语音和视频, Skype 客户都有许多自行支配的不同编解码器, 这些编解码器能够以宽泛的速率和质量对媒体进行编码。例如, 测量表明 Skype 的视频速率从用于低质量会话的低至 30kbps 到用于高质量会话的高至 1Mbps 左右 [Zhang X 2012]。一般而言, Skype 语音质量好于由有线电话系统提供的 “POTS (简单老式电话服务)” 的质量。(Skype 编解码器通常以 16 000 样本/秒或更高速率对语音抽样, 这提供比 POTS 更为丰富的音色, POTS 的抽样率为 8000/秒。) 在默认状态下, Skype 通过 UDP 发送音频和视频分组。然而, 控制分组经 TCP 发送, 并且当防火墙阻挡 UDP 流时, 媒体分组也通过 TCP 发送。Skype 对于经 UDP 发送的语音和视频流使用 FEC 处理丢包恢复。Skype 客户还通过改变视频质量和 FEC 开销, 使它所发送的音频和视频流适应当前的网络情况 [Zhang X 2012]。

Skype 以一些创新方式使用 P2P 技术, 很好地阐述了 P2P 是如何应用于除内容分发和文件共享之外的应用中的。如同即时讯息那样, 主机到主机因特网电话应用的核心内在地应用了 P2P 技术, 因为用户对 (即对等方) 彼此实时通信。但是 Skype 也对两个其他重要功能应用了 P2P 技术, 这两个功能是用户定位和 NAT 穿越。

如图 7-10 所示, Skype 中的对等方 (主机) 组织成为一个等级制覆盖网络, 其中每个对等方分类为超级对等方和普通对等方。Skype 维护一个索引, 该索引将 Skype 用户名映射为当前的 IP 地址 (和端口号)。该索引经过超级对等方分发。当 Alice 要呼叫 Bob 时, 她的客户搜索该分布式索引以决定 Bob 的当前 IP 地址。因为 Skype 协议是专用的, 所以当前并不知道该索引映射是怎样跨越这些超级对等方进行组织的, 尽管采用某种形式的 DHT 组织结构是非常可能的。

P2P 技术也被用于 Skype 中继 (relay) 中, 中继对于创建家庭网络中主机之间的呼叫是有用的。许多家庭网络配置提供通过 NAT 接入因特网, 如第 4 章所讨论的那样。前面讲过 NAT 防止来自家庭网络外部的发起的对家庭网络内部主机的连接。如果两个

Skype 呼叫方都具有 NAT，则存在一个问题，即任一方都不能接受由其他一方发起的呼叫，使得呼叫看起来不可能实现。明智地使用超级对等方和中继很好地解决了这个问题。假设当 Alice 注册进入系统，她被指派了一个非 NAT 的超级对等方并对那个超级对等方发起一个会话。（因为是 Alice 发起了该会话，所以她的 NAT 允许该会话。）这个会话允许 Alice 和她的超级对等方交换控制报文。当 Bob 注册进入系统时发生了同样的事情。此时，当 Alice 要呼叫 Bob，她通知她的超级对等方，超级对等方依次通知 Bob 的超级对等方，Bob 的超级对等方依次通知 Bob 说“Alice 的人呼叫到了”。如果 Bob 接受了该呼叫，这两个超级对等方选择一个第三方非 NAT 超级对等方（即中继对等方），中继对等方的工作是中继 Alice 和 Bob 的数据。Alice 和 Bob 的超级对等方则分别指示 Alice 和 Bob 与该中继发起会话。如图 7-10 所示，Alice 则经过“Alice 到中继”连接向该中继发送语音分组（该连接由 Alice 发起），并且该中继经“中继到 Bob”连接转发这些分组（该连接由 Bob 发起）；从 Bob 到 Alice 的分组反方向地流经相同的两条中继连接。瞧！Bob 和 Alice 有了一条端到端连接，即使他们都不能接受一条源于外部的会话。

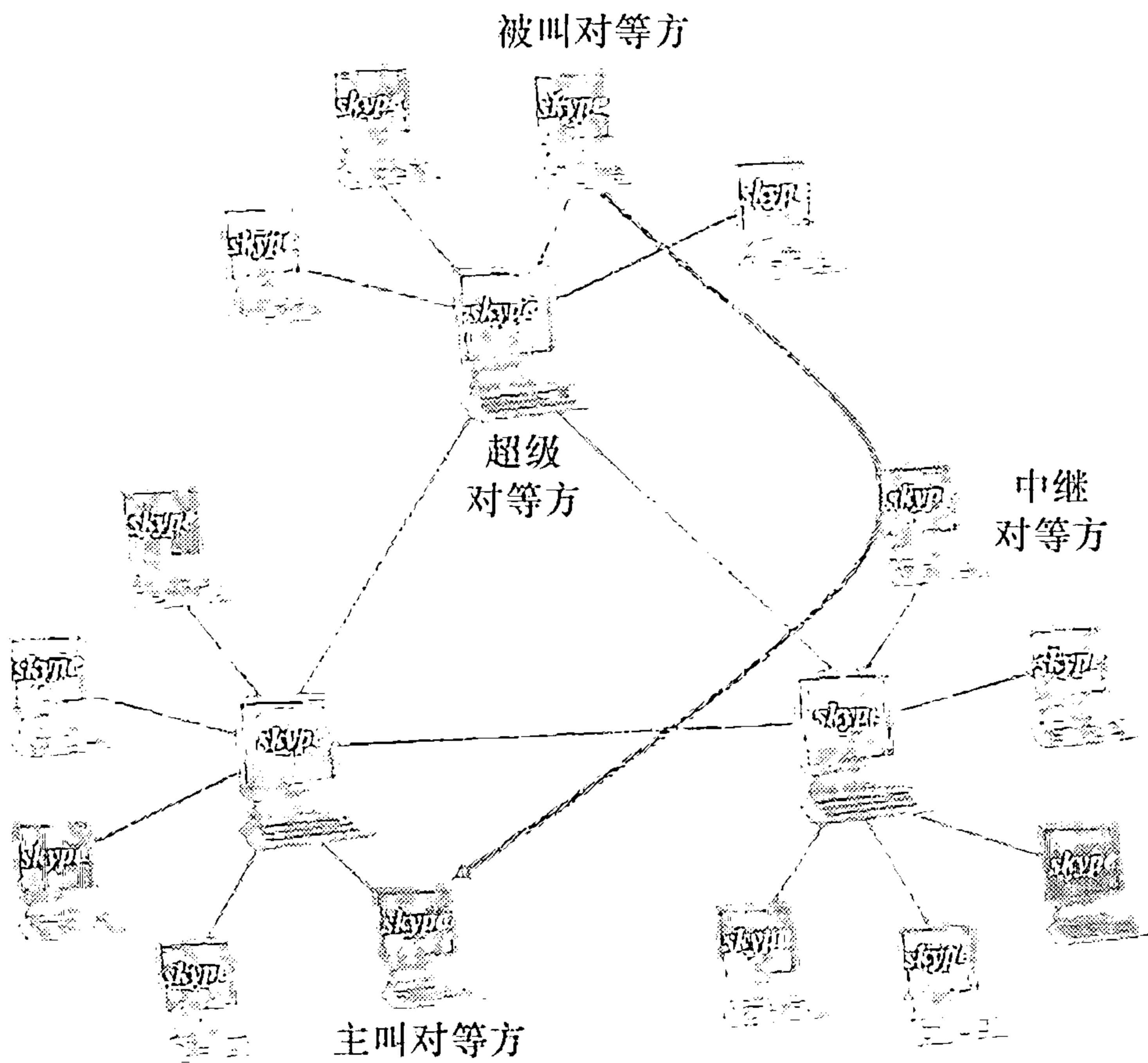


图 7-10 Skype 对等方

到现在为止，我们有关 Skype 的讨论关注涉及两人的呼叫。现在我们观察多方音频会议呼叫。对于 $N > 2$ 个参与者，如果每个用户希望向每个其他 $N - 1$ 个用户发送它的音频流的一个副本，则为支持该音频会议总共 $N(N - 1)$ 个音频流将需要发送到网络中去。为了减少这种带宽使用，Skype 应用了一种明智的发送技术。具体而言，每个用户向会议发起方发送它的音频流。会议发起方将这些音频流结合为一个流（基本上是将所有的音频信号加在一起），然后再向每个其他 $N - 1$ 个参与者发送每个结合流的一个副本。以这种方式，流的数量被减少到 $2(N - 1)$ 条。对普通的两人视频会议，Skype 路由对等方到对等方呼叫，除非需要 NAT 穿越，此时呼叫通过一个非 NAT 对等方中继，如前面所述。对于一个涉及 $N > 2$ 个参与者的视频会议呼叫，由于视频媒体的性质，Skype 不像对语音呼叫那样在一个位置将呼叫结合进一条流中，然后将流向所有参与者重新分发。相反，每个参与者的视频流被路由到一个服务器集群（2011 年该服务器集群位于爱沙尼亚），该集群依次将 $N - 1$ 个其他参与者的 $N - 1$ 条

流中继到每个参与者 [Zhang X 2012]。你可能想知道为什么每个参与者向服务器而不是向每个其他 $N-1$ 其他参与者直接发送其视频流的副本呢？的确，对两种方法而言， $N(N-1)$ 个视频流正由会议中的 N 个参与者共同接收。其原因是，在大多数接入链路中上行链路带宽比下行链路带宽要低得多，上行链路可能不能支持使用 P2P 方法的 $N-1$ 条流。

诸如 Skype、QQ 和 Google Talk 等 VoIP 系统引入了新的隐私性关注。具体而言，当 Alice 和 Bob 经过 VoIP 通信，Alice 能够嗅探到 Bob 的 IP 地址，进而使用地理定位服务 [MaxMind 2012; Quova 2012] 来确定 Bob 的当前位置和 ISP（例如他的工作或家庭 ISP）。事实上，Alice 使用 Skype 能够在呼叫创建期间阻挡特定的分组传输，这样她获得 Bob 当前（比如说每小时）的 IP 地址，而 Bob 并不知道他正被跟踪，并且 Alice 不在 Bob 的接触列表上。此外，从 Skype 发现的 IP 地址能被关联到在 BitTorrent 中发现的 IP 地址，因此 Alice 能够确定 Bob 正在下载的文件 [LeBlond 2011]。此外，通过进行流中分组长度的流量分析，可能部分解密一个 Skype 呼叫 [White 2011]。

7.4 实时会话式应用的协议

实时会话式应用（包括 VoIP 和视频会议）引人入胜并且非常流行。因此标准机构如 IETF 和 ITU 多年来一直忙于（而且要继续忙下去！）苦心推敲这类应用的标准是毫不奇怪的。借助于实时会话式应用的适当标准，各个独立公司正在创造新的能够互相操作的产品。在本节中，我们探讨用于实时会话式应用的 RTP 和 SIP。这两个标准正广泛地应用于工业产品中。

7.4.1 RTP

在前一节中，我们知道 VoIP 应用的发送端在将块传递给运输层之前为它们附加上首部字段。这些首部字段包括了序号和时间戳。因为大多数多媒体网络应用能够利用序号和时间戳，因此有一个包括音频/视频数据、序号、时间戳以及其他潜在有用字段的标准分组结构是方便的。定义在 RFC 3550 中的 RTP 就是这样一个标准。RTP 能够用于传输通用格式，如用于声音的 PCM、ACC 和 MP3，用于视频的 MPEG 和 H.263。它也可以用于传输专用的声音和视频格式。目前，RTP 在许多产品和研究原型中得到广泛实现。它也是其他重要的实时交互协议（如 STP）的补充。

本节我们介绍 RTP。我们也鼓励读者去访问 Henning Schulzrinne 的 RTP 站点 [Schulzrinne-RTP 2012]，该网站提供有关这个主题的很多信息。读者也可以访问 RAT 站点 [RAT 2012]，它记载了使用 RTP 的 VoIP 应用。

1. RTP 基础

RTP 通常运行在 UDP 之上。发送端在 RTP 分组中封装媒体块，然后在 UDP 报文段中封装该分组，然后将该报文段递交给 IP。接收端从 UDP 报文段中提取出这个 RTP 分组，然后从 RTP 分组中提取出媒体块，并将这个块传递给媒体播放器来解码和呈现。

举例来说，考虑使用 RTP 来传输语音。假设语音源采用了 64kbps 的 PCM 编码（也就是采样、量化和数字化）。再假设应用程序在 20ms 块中收集这些编码数据，也就是一个块中有 160 字节。发送端在每个语音数据块的前面加上一个 RTP 首部（RTP header），这个首部包括音频编码的类型、序号和时间戳。RTP 首部通常是 12 字节。音频块和 RTP 首部一起形成 RTP 分组（RTP packet）。然后向 UDP 套接字接口发送该 RTP 分组。在接收端，

应用程序从它的套接字接口收到该 RTP 分组，从 RTP 分组中提取出该音频块，并且使用 RTP 分组的首部字段来适当地解码和播放该音频块。

如果一个应用程序集成了 RTP——而非一个提供负载类型、序号或者时间戳的专用方案，则该应用程序将更容易和其他网络的多媒体应用程序互操作。例如，如果两个不同的公司都开发 VoIP 软件，并且都在它们的产品中集成了 RTP，则希望使用一种 VoIP 产品的用户能够和使用另一种 VoIP 产品的用户进行通信。在 7.4.2 节，我们将看到 RTP 经常和 SIP 一起使用，SIP 是一种因特网电话的重要标准。

应该强调的是，RTP 并不提供任何机制来确保数据的及时交付，或者提供其他服务质量 (QoS) 保证；它甚至不保证分组的交付，或防止分组的失序交付。RTP 封装的东西确仅为端系统所见。路由器不区分携带 RTP 分组的 IP 数据报和不携带 RTP 分组的 IP 数据报。

RTP 允许为每个源（例如一架照相机或者一个麦克风）分配一个它自己的独立 RTP 分组流。例如，对于在两个参与者之间的一个视频会议，可能打开 4 个 RTP 流，即两个流传输音频（一个方向一个），两个流传输视频（也是一个方向一个）。然而，在编码过程中很多流行的编码技术（包括 MPEG1 和 MPEG2）将音频和视频捆绑在单个流中。当音频和视频与编码器捆绑时，每个方向只产生一个 RTP 流。

RTP 分组并非限于单播应用，它们也可以在一对多和多对多的多播树上发送。对于一个多对多的多播会话，所有的会话发送方和源通常使用同样的多播组来发送它们的 RTP 流。在一起使用的 RTP 多播流，例如在视频会议应用中从多个发送方发出的音频和视频流，同属于一个 RTP 会话（RTP session）。

2. RTP 分组首部字段

如图 7-11 所示，4 个主要的 RTP 分组首部字段是有效载荷类型、序号、时间戳和源标识符字段。



图 7-11 RTP 首部字段

RTP 分组中的有效载荷类型字段的长度是 7 比特。对于音频流，有效载荷类型字段用于指示所使用的音频编码类型（例如 PCM、适应性增量调制、线性预测编码）。如果发送方在会话过程中决定改变编码，发送方可以通过该有效载荷类型字段来通知接收方这种变化。发送方可能要通过改变该编码来提高语音质量或者减小 RTP 流比特率。表 7-2 列出了当前 RTP 支持的一些音频有效载荷类型。

表 7-2 RTP 支持的一些音频有效载荷类型

有效载荷类型编号	音频格式	采样速率	速率
0	PCM μ 律	8kHz	64kbps
1	1016	8kHz	4.8kbps
3	GSM	8kHz	13kbps
7	LPC	8kHz	2.4kbps
9	G.722	16kHz	48 ~ 64kbps
14	MPEG 音频	90kHz	—
15	G.728	8kHz	16kbps

对于一个视频流，有效载荷类型用于指示视频编码类型（例如运动 JPEG、MPEG1、MPEG2、H. 261）。发送方也可以在会话期间动态改变视频编码。表 7-3 列出了当前 RTP 支持的一些视频有效载荷类型。

表 7-3 RTP 支持的一些视频有效载荷类型

有效载荷类型编号	视频格式	有效载荷类型编号	视频格式
26	运动 JPEG	32	MPEG1 视频
31	H. 261	33	MPEG2 视频

其他重要的字段如下：

- 序号字段。序号字段长为 16 比特。每发送一个 RTP 分组则该序号增加 1，而且接收方可以用该序号来检测丢包和恢复分组序列。例如，如果应用的接收方收到的 RTP 分组流在序号 86 和 89 之间存在一个间隙，那么接收方则知道分组 87 和 88 丢失了。那么接收方能够设法来隐藏该丢失数据。
- 时间戳字段。时间戳字段长 32 比特。它反映了 RTP 数据分组中的第一个字节的采样时刻。如我们在上一节所见，接收方能够使用时间戳来去除网络中引入的分组时延抖动，提供接收方的同步播放。时间戳是从发送方的采样时钟中获得的。举例来说，对于音频的每个采样周期（例如对于 8kHz 的采样时钟每 125μs 为一个周期）时间戳时钟增加 1；如果该音频应用产生由 160 个编码采样组成的块的话，那么当源激活时，对每个 RTP 分组则时间戳增加 160。即使源未激活，该时间戳时钟也将继续以恒定速率增加。
- 同步源标识符（SSRC）。SSRC 字段长为 32 比特。它标识了 RTP 流的源。通常在 RTP 会话中的每个流都有一个不同的 SSRC。SSRC 不是发送方的 IP 地址，而是当新的流开始时源随机分配的一个数。两个流被分配相同 SSRC 的概率是很小的。如果发生了，这两个源应当选择一个新的 SSRC 值。

7.4.2 SIP

定义在 [RFC 3261；RFC 5411] 中的会话发起协议（Session Initiation Protocol，SIP）是一个开放和轻型的协议，其功能如下：

- 提供了在主叫者和被叫者之间经 IP 网络创建呼叫的机制。它允许主叫者通知被叫者它要开始一个呼叫。它允许参与者约定媒体编码，也允许参与者结束呼叫。
- 提供了主叫者确定被叫者的当前 IP 地址的机制。因为用户可能动态地分配到地址（使用 DHCP），而且因为它们可能有多个 IP 设备，每个都有一个不同的 IP 地址，所以用户不具有单一的、固定的 IP 地址。
- 提供了用于呼叫管理的机制，这些机制包括在呼叫期间增加新媒体流、在呼叫期间改变编码、在呼叫期间邀请新的参与者、呼叫转移和呼叫保持等。

1. 向已知 IP 地址建立一个呼叫

为了理解 SIP 的本质，最好看一个具体的例子。在这个例子中，Alice 在使用 PC，并且她要呼叫 Bob，Bob 也在使用 PC 工作。Alice 和 Bob 的 PC 都配有基于 SIP 的软件来产生和接收电话呼叫。在这个初始的例子中，我们将假设 Alice 知道 Bob PC 的 IP 地址。图 7-12 描述了这个 SIP 呼叫建立的过程。

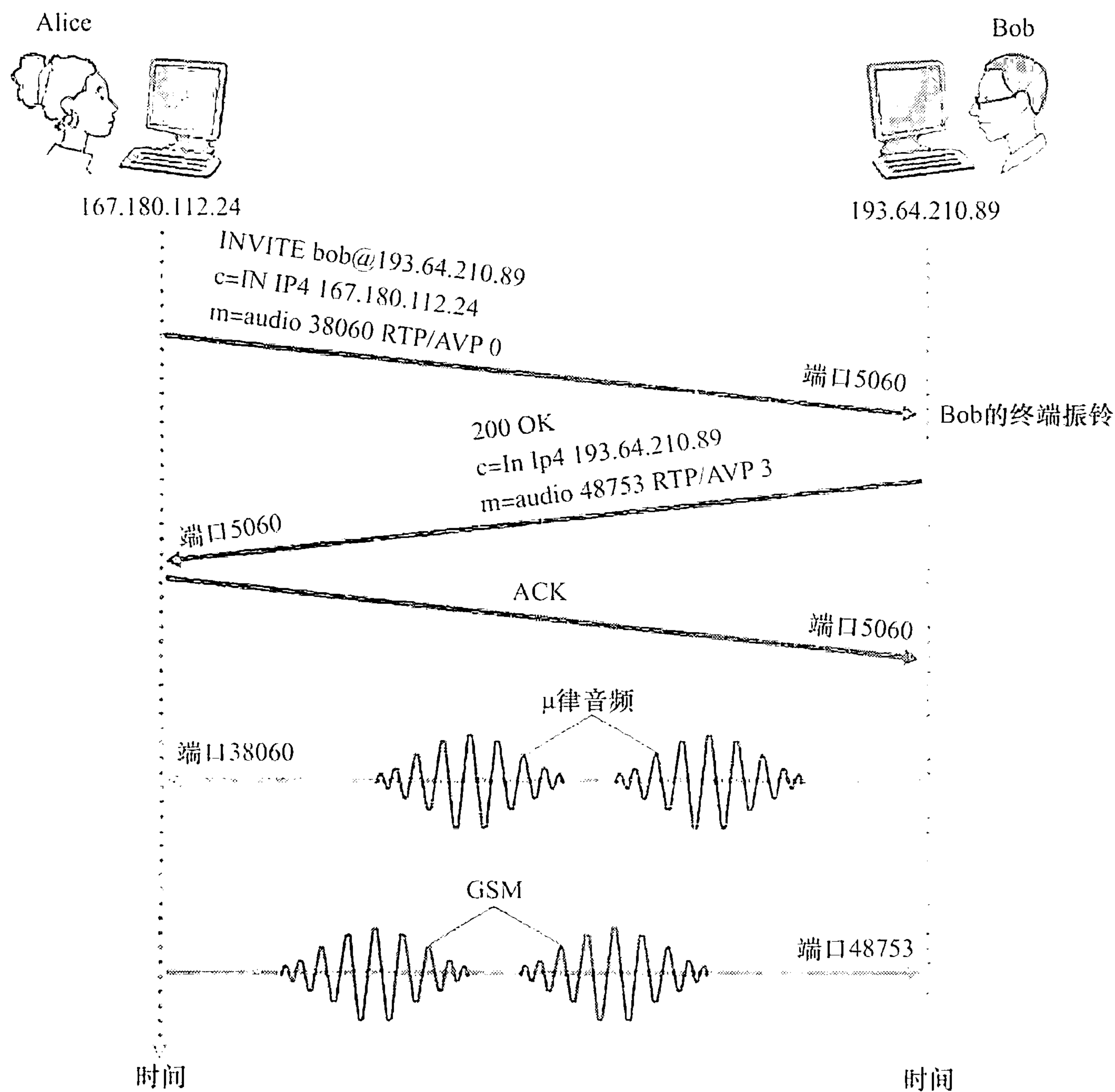


图 7-12 当 Alice 知道 Bob 的 IP 地址时的 SIP 呼叫建立过程

在图 7-12 中，我们看到当 Alice 给 Bob 发送一个 INVITE 报文（这类似于 HTTP 请求报文）时，一个 SIP 会话开始了。该 INVITE 报文通过 UDP 发送给 SIP 的周知端口 5060。（SIP 报文也可以经 TCP 发送。）该 INVITE 报文包括了对 Bob 的标识（bob@193.64.210.89）、Alice 当前 IP 地址的指示、Alice 希望接收的音频的指示（该音频以格式 AVP 0 编码，即 μ 律 PCM 编码，并在 RTP 中封装），以及她希望在端口 38060 接收 RTP 分组的指示。在收到了 Alice 的 INVITE 报文之后，Bob 发送一个 SIP 响应报文（该报文类似 HTTP 的响应报文）。该响应 SIP 报文也被发送到 SIP 端口 5060。Bob 的响应包括一个 200 OK 和他的 IP 地址的指示、他希望接收的编码和分组化，以及音频数据应该发送到达的端口号。注意到在这个例子中，Alice 和 Bob 准备使用不同的音频编码机制：要求 Alice 使用 GSM 来对她的音频编码，而要求 Bob 使用 μ 律 PCM 对他的音频编码。在接收到 Bob 的响应后，Alice 向 Bob 发送 SIP 确认报文。在这个 SIP 事务之后，Bob 和 Alice 可以交谈了。（为了看起来清晰，图 7-12 显示 Alice 在 Bob 之后说话，但是事实上他们通常可以同时进行交谈。）Bob 会根据要求对音频进行编码和分组化，并将音频分组发送给 IP 地址 167.180.112.24 的端口号 38060。Alice 也会根据要求对音频进行编码和分组化，并将音频分组发送给 IP 地址 193.64.210.89 的端口号 48753。

从这个简单的例子我们学到了一些 SIP 的关键特性。首先，SIP 是一个带外协议，即

发送和接收 SIP 报文使用了一个不同于发送和接收媒体数据的套接字。第二，SIP 报文本身是可读的 ASCII，这与 HTTP 报文类似。第三，SIP 要求所有的报文都要确认，因此它能够在 UDP 或者 TCP 上运行。

在这个例子中，我们考虑一下如果 Bob 没有 μ 律 PCM 编解码器用于音频编码将会发生什么情况。在这种情况下，Bob 不用 200 OK 来响应，而可能用一个 606 Not Acceptable（不可接受）来响应，并在报文中列出他能够使用的所有编解码。然后 Alice 从中选择一个编解码，并发送另一个 INVITE 报文，以此通告了已选择的编解码器。Bob 也能够直接通过发送某个拒绝应答代码来直接拒绝该呼叫。（有很多这种代码，包括“busy（忙）”、“gone（离开）”、“payment（付费）”和“forbidden（禁止）”。）

2. SIP 地址

在前面的例子中，Bob 的 SIP 地址是 sip: bob@193.64.210.89。然而，我们希望许多（即使不是大多数）SIP 地址类似于电子邮件地址。例如，Bob 的地址可以是 sip: bob@domain.com。当 Alice 的 SIP 设备发送 INVITE 报文，该报文包括这种类似于电子邮件地址的地址；然后 SIP 的基本设施将该报文转发给 Bob 正在使用的 IP 设备（如我们下面要讨论的那样）。其他可能的 SIP 地址形式可以是 Bob 过去的电话号码或者只是 Bob 的名字/中间名/姓氏（假设它是唯一的）。

SIP 地址的一个有趣特点是它们能够被包括在 Web 页面中，就像人们的电子邮件地址用 mailto URL 形式包含在 Web 页面中那样。例如，假设 Bob 有个人主页，并且他要为这个主页的访问者提供一个呼叫他的方法。于是他可能只是在主页中包括该 URL sip: bob@domain.com。当访问者点击该 URL，访问者设备中的 SIP 应用将启动，并向 Bob 发送 INVITE 报文。

3. SIP 报文

在这个简短的 SIP 介绍中，我们无法包括所有 SIP 报文类型和首部，而只是简要地浏览一下 SIP INVITE 报文，以及少数通用的首部行。我们再假设 Alice 要对 Bob 发起 VoIP 呼叫，此时 Alice 只知道 Bob 的 SIP 地址 bob@domain.com，并不知道 Bob 正在使用的设备的 IP 地址。那么她的报文可能看起来有些像下面这个：

```
INVITE sip:bob@domain.com SIP/2.0
Via: SIP/2.0/UDP 167.180.112.24
From: sip:alice@hereway.com
To: sip:bob@domain.com
Call-ID: a2e3a@pigeon.hereway.com
Content-Type: application/sdp
Content-Length: 885
```

```
c=IN IP4 167.180.112.24
m=audio 38060 RTP/AVP 0
```

这个 INVITE 行包括 SIP 的版本，这与 HTTP 请求报文一样。任何时候 SIP 报文通过一个 SIP 设备（包括产生该报文的设备）时，它附加上一个 Via 首部来指示该设备的 IP 地址。（我们不久将看到通常 INVITE 报文在到达被叫者的 SIP 应用之前会通过很多 SIP 设备。）与电子邮件报文类似，SIP 报文包括一个 From 首部行和一个 To 首部行。该报文包括一个 Call-ID（呼叫标识符），它唯一地标识该呼叫（类似电子邮件中的报文 ID）；包括一个 Content-Type（内容类型）首部行，定义用于描述包含在 SIP 报文中的内容的格式；还包括 Content-Length（内容长度）首部行，提供报文中内容的字节长度。最后，在一个回

车和换行之后，该报文包含内容部分。在这个例子中，内容提供了有关 Alice 的 IP 地址和 Alice 要如何接收该音频的信息。

4. 名字翻译和用户定位

在图 7-12 的例子中，我们假设 Alice 的 SIP 设备知道能够联系到 Bob 的 IP 地址。但是这种假设是很不真实的，不仅因为 IP 地址通常是通过 DHCP 动态分配的，而且 Bob 可能有多个 IP 设备（例如，在家里、工作中和汽车里有不同的设备）。因此现在我们假设 Alice 只知道 Bob 的电子邮件地址 bob@domain.com，而且对基于 SIP 的呼叫使用同样的地址。在这种情况下，Alice 需要获得用户 bob@domain.com 正在使用的设备的 IP 地址。为了获得它，Alice 创建一个 INVITE 报文，它以 INVITE bob@domain.com SIP/2.0 开始，并将该报文发送给一个 SIP 代理（SIP proxy）。该代理将以一个 SIP 回答来响应，该回答中也可能包含 bob@domain.com 正在使用的设备的 IP 地址。该回答也可以选择包括 Bob 的语音信箱的 IP 地址，或者可能包括一个 Web 页面的 URL（上面写着“Bob 在睡觉，请不要打扰！”）。代理返回的结果也可能取决于呼叫者：如果呼叫是 Bob 的妻子发出的，它可能接受该呼叫，并提供 IP 地址；如果呼叫是 Bob 的岳母发出的，它可能用指向“我正在睡觉”的 Web 页面的 URL 来响应。

现在，你可能想知道这个代理服务器是怎样确定 bob@domain.com 现在的 IP 地址的？为了回答该问题，我们需要讲一下另一个 SIP 设备，即 SIP 注册器（SIP registrar）。每个 SIP 用户都有一个相关联的注册器。任何时候用户在设备上发起 SIP 应用时，该应用给注册器发送一个 SIP 注册报文，通知注册器它现在的 IP 地址。例如，当 Bob 在他的 PDA 上发起 SIP 应用时，该应用将发送一个类似于下述内容的报文：

```
REGISTER sip:domain.com SIP/2.0
Via: SIP/2.0/UDP 193.64.210.89
From: sip:bob@domain.com
To: sip:bob@domain.com
Expires: 3600
```

Bob 的注册器跟踪 Bob 现在的 IP 地址。无论何时 Bob 切换到一个新的 SIP 设备时，该新设备将发送一个新的注册报文，指示该新的 IP 地址。如果 Bob 长时间使用同样的设备，该设备将发送刷新注册报文，指示最近发送的 IP 地址仍然有效。（在上面的例子中，需要每隔 3600 秒发送刷新报文来维护在注册器中的地址。）值得注意的是注册器和 DNS 权威名字服务器类似：DNS 服务器将固定的用户名翻译到固定的 IP 地址；SIP 注册器把固定的人识别标志（例如 bob@domain.com）翻译为一个动态的 IP 地址。SIP 注册器和 SIP 代理通常运行在同一台主机上。

现在我们检查一下 Alice 的 SIP 代理服务器是如何获得 Bob 当前的 IP 地址的。从上面的讨论我们看到，代理服务器只需要转发 Alice 的 INVITE 报文给 Bob 的注册器/代理即可。然后注册器/代理将该报文转发给 Bob 现在的 SIP 设备。最后，在已收到了 Alice 的 INVITE 报文之后，Bob 可以向 Alice 发送一个 SIP 应答。

举例来说，考虑图 7-13，其中正在 217.123.56.89 上工作的 jim@umass.edu 要对正在 197.87.54.21 上工作的 keith@upenn.edu 发起 IP 语音（VoIP）会话。采用下面的步骤：①Jim 向 umass 的 SIP 代理发送 INVITE 报文。②该代理在 SIP 注册器 upenn.edu 上进行 DNS 查询（在图中没有显示），并将该报文转发给注册服务器。③因为 keith@upenn.edu 没有在注册器 upenn 上注册，upenn 注册器发送一个重定向应答，指示它应该试一试

keith@eurecom.fr。④umass 的代理向 eurecom 的 SIP 注册器发送 INVITE 报文。⑤eurecom 注册器知道 keith@eurecom.fr 的 IP 地址，并将 INVITE 报文转发给正运行 Keith 的 SIP 客户的主机 197.87.54.21。⑥~⑧通过注册器/代理把 SIP 响应返回给 217.123.56.89 上的 SIP 客户。⑨媒体直接在两个客户之间发送。（还有一个 SIP 确认报文，图中没有画出来。）

我们有关 SIP 的讨论集中在语音呼叫的呼叫发起方面。SIP 通常是一个发起和结束呼叫的信令协议，它能够用于视频会议呼叫和基于文本的会话。事实上，SIP 已经成为许多即时讯息应用的基本组件。我们鼓励希望学习更多有关 SIP 知识的读者访问 Henning Schulzrinne 的 SIP Web 站点 [Schulzrinne-SIP 2012]。特别是，你在这个站点上会发现 SIP 客户和服务器的开源软件 [SIP Software 2012]。

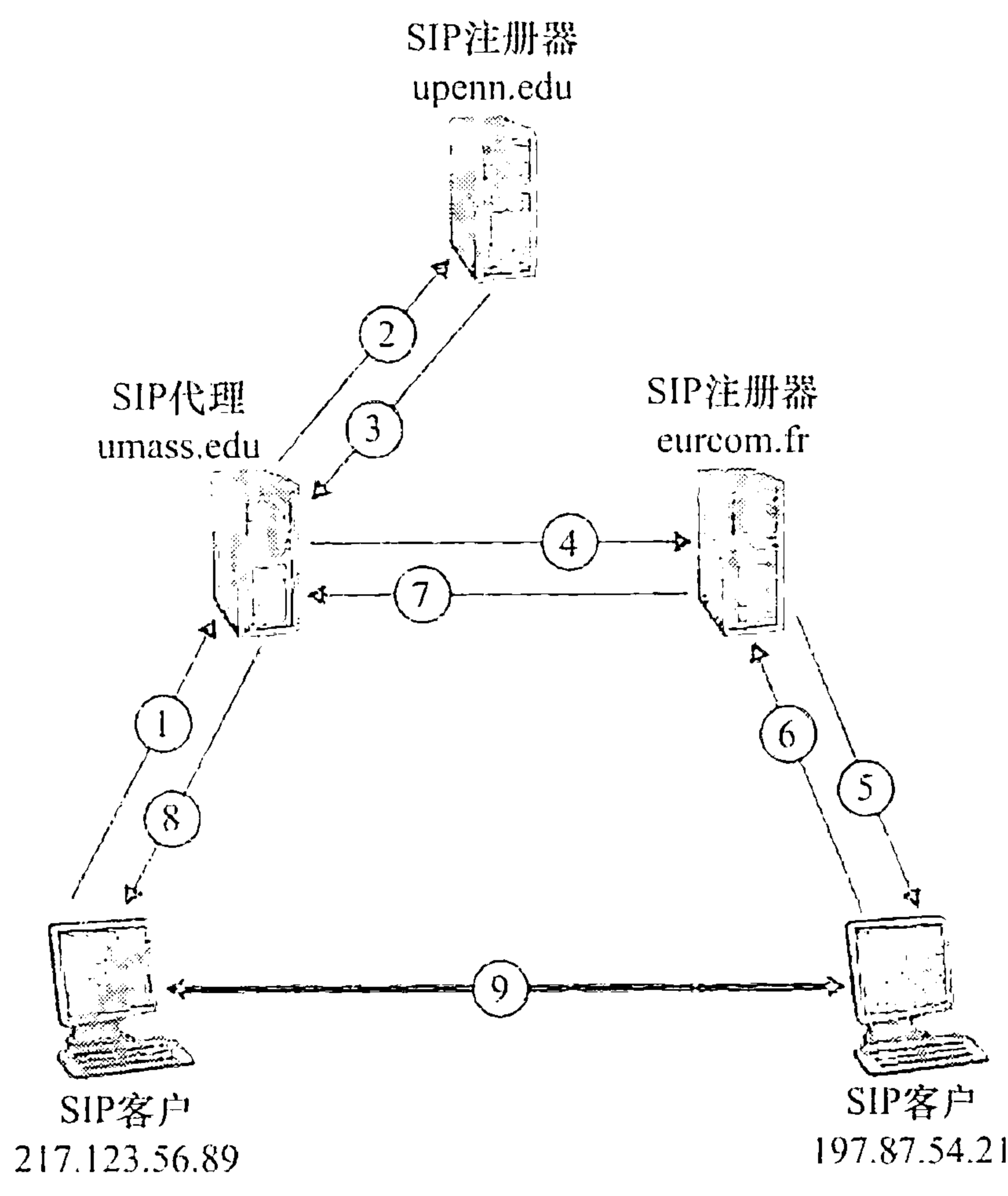


图 7-13 涉及 SIP 代理与注册器的会话发起

7.5 支持多媒体的网络

在 7.2 ~ 7.4 节中，我们学习了诸如客户缓存、预取、对可用带宽的适应性媒体质量、适应性播放和丢包缓解技术等应用级机制如何用于多媒体应用，以改善多媒体应用的性能。我们也学习了内容分发网和 P2P 覆盖网络如何用于提供系统级的交付多媒体内容的方法。这些技术和方法都被设计用于今天的尽力而为因特网。的确，现在因特网得到应用就是因为它只提供单一的、尽力而为类型的服务。但是作为计算机网络的设计者，我们禁不住要问：网络（而不是应用程序或者仅应用级的基础设施）是否可以提供支持多媒体内容交付的机制。如我们很快将看到的那样，答案当然是肯定的！但是我们也将看到，许多这样的新网络级机制还没有得到广泛部署。这可能是由于它们的复杂性和下列事实所致：应用级基础设施以及尽力而为服务和适当定制的网络资源（例如带宽）的确能够提供“足够好的”（即使不总是尽善尽美的）端到端多媒体交付服务。

表 7-4 总结了能够对多媒体应用提供网络级支持的三种宽泛的方法。

表 7-4 支持多媒体应用的三种网络级方法

方法	粒度	保证	机制	复杂性	当前部署
尽可能利用尽力而为服务	公平处理所有流量	无或者软	应用级支持，CDN，覆盖网络，网络级资源供给	最小	无处不在
区分服务	不同类型的流量处理不同	无或者软	分组标识，监管，调度	中等	某些
每连接服务质量（QoS）保证	每个源到目的地流处理不同	一旦流被准入，软或者硬	分组标识，监管，调度，呼叫准入和信令	高	很少

- 尽可能利用尽力而为服务。我们在 7.2 ~ 7.4 节学习的应用级机制和基础设施能够成功地用于定制良好的网络——该网络中偶然出现丢包和过大的端到端时延。当预见到需求增加，ISP 部署额外的带宽和交换能力以持续确保满意的时延和丢包性能 [Huang 2005]。我们将在 7.5.1 节中进一步讨论网络定制（network dimensioning）。
- 区分服务。自因特网的早期，就已经设想不同类型的流量（例如，在 IPv4 分组首部中服务类型字段所指示的类型）能够由不同类型的服务所提供，而不是单一的“以不变应万变”的尽力而为服务。使用区分服务，当两类流量在一台路由器中排队时，一种类型的流量可以给定严格的优于另一种类型的流量的优先权。例如，属于实时会话式应用的分组由于其严格的时延限制，可能会给定优于其他分组的优先权。在网络中引入区分服务将要求一些用于分组标记（指示一个分组的服务类型）、分组调度和其他方面的新机制。我们将在 7.5.2 节中涉及区分服务以及实现这种服务的新网络机制。
- 每连接服务质量（QoS）保证。使用每连接 QoS 保证，每个应用的实例显式地预约端到端带宽，并因此具有确保的端到端性能。硬保证（hard guarantee）意味着应用将必定接收到它所请求的服务质量。软保证（soft guarantee）意味着应用将以高概率接收到它所请求的服务质量。例如，如果某用户要从主机 A 向主机 B 进行 VoIP 呼叫，该用户的 VoIP 应用在两台主机之间沿着路径在每条链路上显式地预留带宽。但是，允许应用做预约和请求网络同意该预约，这需要一些大的变化。首先，我们需要一个协议来代表应用程序，从发送方到其接收方沿路径预约链路带宽。第二，在路由器队列中将需要新的调度策略，使每连接带宽预约能够兑现。最后，为了进行预约，应用程序必须向网络给出描述来说明它们希望发送进网络的流量，并且网络将需要监管每个应用程序的流量以确保它遵守这个描述。当这些机制结合时，在主机和路由器中要求新的和复杂的软件。因为每连接 QoS 保证服务尚未见到大规模部署，我们将仅在 7.5.3 节中简要地涉及这些机制。

7.5.1 定制尽力而为网络

从根本上说，支持多媒体应用的困难是由其严格的性能要求引起的，即低的端到端分组时延、时延抖动和丢包，而事实是，无论何时网络变得拥塞，都将出现较大的分组时延、时延抖动和丢包。改善多媒体应用质量的第一种方法就是“在问题上砸钱”，因此直接避免资源竞争即可。这种方法常用于解决有关资源受限的任何问题。在网络多媒体的场合，这意味着在整个网络中提供充足的链路带宽，使网络拥塞及其导致的分组时延和丢失决不会（或仅非常少地）出现。具有充足的链路带宽，分组将很快地通过今天的因特网而没有排队时延或丢失。从许多方面看，这是一种理想的情况：完美地执行多媒体应用，用户是幸运的，所有要求都能够满足而不改变因特网尽力而为的体系结构。

当然，问题是为实现这种极乐世界提供多大容量才是“充足的”，以及提供“充足的”带宽的成本从 ISP 的商业角度来说是否实际。在一个给定拓扑中为网络链路提供多大容量以取得给定水平的性能的问题常被称为带宽供给（bandwidth provisioning）。如何设计一个网络拓扑（其中放置一些路由器，如何用链路互联这些路由器，并为链路分配容量）以取得给定水平的端到端性能这个更为复杂的问题常被称为网络定制（network dimensioning）。带宽供给和网络定制都是复杂的专题，它们远超过了本教科书的范围。然而，我们这里注

意到，为了预测两个网络端点之间的应用级性能，必须处理下列问题，并因此提供充足的容量来满足应用的性能要求。

- 网络端点之间的流量要求模型。这些模型可能需要定义在呼叫层次（例如，用户“到达”网络并启动端到端应用）和分组层次（例如，由进行中的应用所产生的分组）。注意负载可能随着时间而变化。
- 定义良好的性能要求。例如，为支持诸如会话式多媒体应用等时延敏感的流量，其性能要求可能是：分组的端到端时延大于最大可容忍时延的概率要小于某个很小的值 [Fraleigh 2003]。
- 对给定的负载模型预测端到端性能的模型，以及求出最小成本带宽分配（该带宽分配将导致满足所有用户的需求）的技术。这里，研究人员正忙于研发能够量化给定负载下的性能的性能模型，以及能求出满足性能要求的最小成本带宽分配的优化技术。

假定今天尽力而为的因特网能够（从技术角度讲）以适当的性能水平支持多媒体流量（如果它被定制成这样的话），自然的问题是为什么今天的因特网满足不了这样的要求。答案基本上在经济上和组织上的原因。从经济角度看，用户将愿意向其 ISP 支付足够多的费用，使 ISP 安装充足的带宽经尽力而为的因特网来支持多媒体应用吗？组织问题也许更为令人气馁。注意到在两个多媒体端点之间的端到端路径将通过多个 ISP 的网络。从组织角度看，这些 ISP 将愿意合作（也许以收入共享方式）以确保端到端路径被适当地定制来支持多媒体应用吗？对这些经济和组织问题的展望，参见 [Davies 2005]。对于供给第一层主干网络以支持时延敏感流量的展望，参见 [Fraleigh 2003]。

7.5.2 提供多种类型的服务

也许对今天因特网中的以不变应万变的尽力而为服务而言，一种最简单的强化是将流量划分为多种类型，并为这些不同类型的流量提供不同等级的服务。例如，某 ISP 可能要为时延敏感的 VoIP 或电信会议流量比电子邮件或 HTTP 等弹性流量提供更高的服务类型（并对该服务收取更高的费用！）。另一种做法是，ISP 可能直接向愿意对这种改进服务支付更多费用的顾客提供更高质量的服务。一些住宅有线接入 ISP 和蜂窝无线接入 ISP 已经采用了这样的梯次等级服务，即铂金卡服务用户比金卡服务用户或银卡服务用户享有更好的服务性能。

我们都从日常生活中熟悉了不同类型的服务，如航班上头等舱乘客比公务舱乘客得到更好的服务，公务舱乘客又比经济舱乘客得到更好的服务；VIP 在活动中能够立即进入，而所有其他人都必须排队等待；在某些国家中老年人被尊重，提供了荣誉座位和最精细的食物。重要的是注意到在聚合流量中（即在多种流量类型中而不是单个连接中）提供了这种有差别的服务。例如，所有头等舱乘客被一视同仁（没有哪个头等舱乘客得到了比其他头等舱乘客更好的服务），就像所有的 VoIP 分组在网络中得到了相同的对待，与它们所属的特定的端到端连接无关。如我们将见到的那样，通过处理少量的流量聚合，而不是大量的单个连接，提供好于尽力而为服务所需要的新型网络机制能够保持相对简单。

早期因特网设计者的心中清晰地具有这种多种类型服务的概念。回想在图 4-13 中 IPv4 首部中的服务类型（ToS）字段。IEN123 [ISI 1979] 描述也呈现在 IPv4 数据报的原型中的 ToS 字段时说：“服务类型 [字段] 提供了所希望的服务质量的抽象参数的指示。